

Four-round Statistical Non-malleable Zero-knowledge

Susumu Kiyoshima

NTT Social Informatics Laboratories
susumu.kiyoshima@ntt.com

September 30, 2025

Abstract

We present a 4-round statistical non-malleable zero-knowledge (NMZK) argument in the plain model under standard hardness assumptions. Our construction can be based on any collision-resistant hash function and injective one-way function, and it guarantees simulation extractability in the delayed-input one-many setting. Before this work, 4-round constructions were known for computational NMZK but not for statistical NMZK.

Contents

1	Introduction	3
1.1	Our Result	4
1.2	Additional Related Work	4
2	Technical Overview	5
2.1	Overall Approach	5
2.2	Constant-round Construction	6
2.3	4-round Construction	10
3	Preliminaries	10
3.1	Notations and Conventions	10
3.2	Interactive Proofs/Arguments	11
3.3	Adaptive-input Interactive Proofs/Arguments	11
3.4	Statistical Non-malleable Zero-knowledge	13
3.5	Synchronous Non-malleable Commitments	14
4	Tool: Special-purpose WIPOK System	16
5	Our 4-round SNMZK Argument	20
5.1	Protocol Description	20
5.2	Proofs of Completeness and Soundness	21
5.3	Description of Simulator-Extractor	23
5.4	Running Time of Simulator-Extractor	25
5.5	Statistical Simulation Property of Simulator-Extractor	25
5.6	Witness Extractability of Simulator-Extractor	26
A	Proof of Theorem 3	36
A.1	Preliminaries	36
A.2	Protocol Description	38
A.3	Security proof	38
B	Full Description of Hybrid Simulator-extractor $\mathcal{SE}'$	42
C	Proof of Claim 4	43

1 Introduction

Non-malleable zero-knowledge. *Non-malleable zero-knowledge* (NMZK) [DDN00] is a notion of zero-knowledge defined in man-in-the-middle settings. In the most basic setting, NMZK is defined against an adversary that interacts with an honest prover and an honest verifier, and the goal is to guarantee that the interaction with the prover does not help the adversary prove a related statement to the verifier. In a work by Pass and Rosen [PR08b], this goal is formalized by requiring the existence of a *simulator-extractor*, which simulates the adversary’s view while also extracting a witness for the statement that the adversary proves. The rationale behind this formalization is that given such a simulator-extractor, a stand-alone simulator can prove the same statement as the man-in-the-middle adversary by using the extracted witness.

A central topic that has been extensively studied about NMZK is the round complexity. Regarding the asymptotic round complexity, the initial construction by Dolev, Dwork, and Naor [DDN00] has logarithmic round complexity, and Barak [Bar02] achieved asymptotically optimal round complexity by providing a constant-round construction using sub-exponentially hard collision-resistant hash functions. Soon after, Pass and Rosen [PR08b] gave a constant-round construction using standard (polynomially hard) collision-resistant hash functions. Finally, Goyal [Goy11] and Lin and Pass [LP15] gave constant-round constructions using one-way functions. Regarding the exact round complexity, Goyal et al. [GRRV21] gave a 4-round construction using one-way functions. (Having 4 rounds is optimal even for non-NM zero-knowledge proof/arguments in the case of black-box zero-knowledge [GK96].) Ciampi et al. [COSV17c] gave a 4-round construction in the *delayed-input setting* using collision-resistant hash functions. (In the delayed-input setting, the statement to be proven is determined in the last round of the protocol.) Finally, Ciampi et al. [COSV17a] gave a 4-round construction in the delayed-input, *one-many* setting using one-way functions. (In the one-many setting, the adversary interacts with a single prover and polynomially many verifiers.)

Statistical non-malleable zero-knowledge. *Statistical non-malleable zero-knowledge* (SNMZK) is a version of NMZK where the simulated view must be statistically close to the real view.¹ It naturally inherits strong security guarantees from statistical ZK in the stand-alone setting. (For example, security is not compromised even if the underlying hardness assumptions are broken after the protocol executions [MU10].²)

Statistical simulation offers an additional benefit in man-in-the-middle settings. As a motivating example, consider a commitment scheme that has the following commit-and-prove structure: the sender commits to a message m by using a (possibly malleable) non-interactive commitment and then proves knowledge of its valid opening by using an NMZK argument.³ A natural question to ask is whether NMZK guarantees non-malleability for this commitment scheme. That is, the question is when a man-in-the-middle adversary receives a commitment to a message m , whether NMZK prevents the adversary from committing to a related message such as $m + 1$. At first glance, one might expect the answer to be yes based on the following reasoning.

1. Suppose we remove the proof part from the “left commitment” but still require the adversary to complete the proof part in the “right commitment” (see Figure 1). Since the non-interactive commitment of the left commitment is not guaranteed to be non-malleable, the adversary might maul it so that the non-interactive commitment of the right commitment is a commitment to a related message. However, in this case, the adversary cannot complete the proof part of the right commitment since it does not know the message committed to in the non-interactive commitment. (This part can be formalized by relying on the non-interactiveness of the commit part and the argument-of-knowledge property of the proof part.)
2. Then, since receiving an NMZK argument is not helpful in generating a new NMZK argument, the adversary should fail to complete the proof part of the right commitment even given the proof part of the left commitment.

¹The man-in-the-middle adversary is restricted to a probabilistic polynomial-time machine, and only the distinguisher is computationally unbounded.

²To break computational soundness, cheating provers need to break the underlying hardness assumptions during protocol executions.

³Note that the above “simulation extractability”-type definition of NMZK implies the standard argument-of-knowledge property.

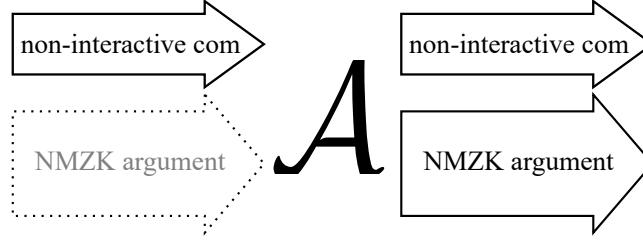


Figure 1: Commit-and-prove protocol with the proof part being removed in the left commitment.

Unfortunately, as observed by Pass and Rosen [PR08b, Section 6.1], the second part of the above reasoning does not work when NMZK only guarantees computational simulation. To see this, suppose the adversary adaptively selects the non-interactive commitment of the right commitment based on the proof part of the left commitment. Then, since the message committed to by the adversary cannot be efficiently recovered, computational simulation does not rule out the possibility that giving an NMZK argument to the adversary could noticeably change the message that the adversary commits to. As a result, in the case of computational NMZK, it is unclear whether the above commitment scheme is non-malleable. Fortunately, as also noted by Pass and Rosen [PR08b], this problem disappears in the case of SNMZK. Indeed, statistical simulation guarantees that giving an NMZK argument to the adversary cannot noticeably change the message that the adversary commits to.⁴

It is known that asymptotically optimal round complexity can be achieved even in the case of SNMZK. Indeed, the aforementioned constant-round construction by Pass and Rosen [PR08b] guarantees SNMZK, and their construction can be extended to satisfy SNMZK in the one-many setting [PR08a]. However, the exact round complexity of SNMZK arguments remains largely unexplored. In particular, whether 4-round SNMZK arguments can be obtained under standard cryptographic assumptions is unknown.

1.1 Our Result

We present the following result regarding 4-round SNMZK arguments.

Theorem (informal). *Assume the existence of collision-resistant hash functions and injective one-way functions. Then, there exists a statistical non-malleable zero-knowledge argument in the delayed-input, one-many setting.*

This result is obtained using black-box simulation techniques. Thus, like prior results on 4-round computational NMZK [GRRV21, COSV17c, COSV17a], this result matches the 4-round lower bound of black-box ZK. An interesting question left open by this result is whether a 4-round SNMZK argument can be constructed from collision-resistant hash functions (without additionally relying on injective one-way functions).

1.2 Additional Related Work

Let us review additional related work that studies NMZK in the plain model (i.e., the model where trusted setups like common reference strings are unavailable).

Concurrent non-malleable zero-knowledge (concurrent NMZK) [BPS06] is a version of NMZK where man-in-the-middle adversaries can interact with polynomially many provers and verifiers. Concurrent NMZK has been extensively studied [BPS06, OPV10, LPTV10, LP11b, Ven14], and there even exist works that achieve *statistical concurrent non-malleable zero-knowledge* [OOR⁺14, Kiy20]. However, the exact round complexity of concurrent NMZK remains largely unexplored. A major obstacle is that even obtaining constant-round

⁴We comment that this application to non-malleable commitment schemes is provided as an illustrative example of how statistical simulations offer an additional benefit. (It is now well-known that non-malleable commitment schemes can be constructed without using NMZK.) Essentially, what this example demonstrates is that statistical simulations are useful in applications where we need to argue about some statistical properties of the statements proven by NMZK arguments.

concurrent (non-NM) zero-knowledge arguments currently requires strong hardness assumptions (e.g., those on which the existence of indistinguishability obfuscation can be based) [CLP13, CLP15, PPS15, FKP19].

Some recent works have investigated NMZK with practical efficiency [KLP22, BCO⁺24]. This paper focuses on establishing a feasibility result and does not aim to optimize practical efficiency.

2 Technical Overview

2.1 Overall Approach

We begin by outlining the main challenge in constructing 4-round SNMZK arguments. Recall that in the definition of NMZK, a simulator-extractor must simulate the man-in-the-middle adversary’s view while also extracting a witness for the statement that the adversary proves.⁵ We call the interaction between the honest prover and the adversary the *left interaction*, and the interaction between the adversary and the honest verifier the *right interaction*.

The main tool used in the existing 4-round computational NMZK arguments [GRRV21, COSV17c, COSV17a] is *non-malleable (NM) commitment schemes* [DDN00]. At a high level, NM commitment schemes guarantee that when a man-in-the-middle adversary interacts with an honest sender and an honest receiver, the value committed to by the adversary is independent of the value committed to by the honest sender. In this technical overview, we adopt the following (slightly stronger) CCA-style definition: any adversary cannot break the hiding property even given access to computationally unbounded “committed-value oracle” (which interacts with the adversary as a receiver and then returns the adversary the committed value of the commitment that the adversary provides).

What is common in the existing 4-round computational NMZK arguments is that NM commitments are made by the prover. In particular, in the existing 4-round computational NMZK arguments, NM commitments are made by the prover in a way that different values are committed in the real execution and the simulation. For example, in the constructions of [GRRV21, COSV17c], a witness for the statement is committed to in the real execution while a junk value (such as an all-0 string) is committed to in the simulation. At a very high level, when the prover makes NM commitments in this way, NMZK can be argued as follows. Suppose a man-in-the-middle adversary proves the validity of a statement in the right interaction by committing to a valid witness in an NM commitment. Then, the non-malleability of the NM commitment guarantees that when the left interaction is switched to simulation (and therefore the committed value of the NM commitment is switched to a junk value in the left interaction), the adversary continues to commit to a valid witness in the right interaction. Thus, a desired simulator-extractor can be obtained by considering a simulator that extracts the committed value of the NM commitment in the right interaction.

A challenge we face is that using NM commitment schemes in this way inherently prevents the simulation from being statistically indistinguishable. Specifically, the problem is that NM commitment schemes must be statistically binding by definition.⁶ When a different value is committed in the real execution and the simulation, a computationally unbounded distinguisher can easily distinguish them by extracting the committed values of the NM commitments by brute force.

Fortunately, this problem has already been addressed in the context of asymptotically round-efficient statistical concurrent NMZK arguments [Kiy20]. At a high level, the idea is to have the verifier make NM commitments (rather than having the prover do so).⁷ When only the verifier makes NM commitments, the statistical binding of the NM commitments is unproblematic for statistical simulation. Furthermore, by having the verifier make NM commitments in a trapdoor generation phase (where a trapdoor is set up for simulation), we can guarantee that the left and right interactions have independent trapdoors. This property can be used to obtain a desired

⁵In this overview, we focus on the one-one NMZK setting (where the adversary interacts with an honest prover and an honest verifier) rather than the one-many NMZK setting (where the adversary interacts with an honest prover and polynomially many honest verifiers).

⁶Non-malleability is not well-defined unless the value committed to by the adversary is uniquely determined.

⁷The idea of having the verifier make NM commitments has also been used for computational NMZK and a related primitive, for different purposes from ours [JP14, CLP20, BCO⁺24].

simulator-extractor by showing that any man-in-the-middle adversary cannot cheat in the right interaction even when the left interaction is simulated using the trapdoor.

Naturally, our approach to 4-round SNMZK involves combining this technique with those developed for 4-round computational NMZK arguments.

2.2 Constant-round Construction

First, we describe our construction without optimizing its round complexity. The starting point is the well-known approach by Feige and Shamir [FS90a] for round-efficient ZK arguments. Recall that, in the case of statistical ZK, the Feige–Shamir approach can be implemented using following three components: a one-way permutation (OWP), a witness-indistinguishable proof of knowledge (WIPOK), and a statistically witness-indistinguishable argument of knowledge (SWIAOK).⁸ Since our approach is to have the verifier make NM commitments, we consider combining the first two components with NM commitment schemes. Concretely, we consider the following construction for statement $x \in \mathbf{L}$.

1. **Trapdoor generation.** The verifier uses an NM commitment scheme twice to commit to two random strings, r_0 and r_1 . Let $i\text{-}\tau_0$ and $i\text{-}\tau_1$ be the resulting commitments. Let dec_0 and dec_1 be the corresponding decommitments.
2. **Verifier proof.** The verifier uses the NM commitment scheme twice to commit to dec_0 and dec_1 , resulting in two commitments, $o\text{-}\tau_0$ and $o\text{-}\tau_1$. Then, the verifier uses a WIPOK system to prove knowledge of $\beta \in \{0, 1\}$ and $\text{dec} \in \{0, 1\}^*$ such that (i) dec is a valid decommitment of $i\text{-}\tau_\beta$ and (ii) $o\text{-}\tau_\beta$ is a correct commitment to dec . (The witness used by the verifier is chosen randomly from $(0, \text{dec}_0)$ or $(1, \text{dec}_1)$.)
3. **Prover proof.** The prover uses an SWIAOK system to prove $x \in \mathbf{L}$ or knowledge of a valid decommitment of $i\text{-}\tau_b$ for some $b \in \{0, 1\}$.

In the following, we call the NM commitments in the trapdoor generation phase the *inner NM commitments*, and the NM commitments in the verifier proof phase the *outer NM commitments*.

2.2.1 Case 1: NMZK in asynchronous message scheduling.

As a warm-up, let us analyze SNMZK in the setting where the messages in the left and right interactions are scheduled as shown in Figure 2 (left). Consider a natural simulator-extractor that (i) simulates the left interaction using the extractor of the WIPOK system to obtain a witness for the SWIAOK system and (ii) extracts a witness for the statement in the right interaction using the extractor of the SWIAOK system. Statistical indistinguishability of the simulation can be shown in the standard manner by relying on the knowledge soundness of the WIPOK system and the statistical WI of the SWIAOK system. Regarding witness extractability, assume for contradiction that the simulator-extractor fails to extract a witness for the statement in the right interaction with non-negligible probability. Then, the knowledge soundness of the SWIAOK system guarantees that a valid decommitment of one of the two inner NM commitments is extracted with non-negligible probability. In addition, the witness indistinguishability of the WIPOK system guarantees that $\exists \tilde{\beta}^* \in \{0, 1\}$ s.t. the following event, BAD, occurs with non-negligible probability.

- **Event BAD:** The WIPOK system in the right interaction is completed using the decommitment of the $\tilde{\beta}^*$ -th inner NMCom commitment as the witness, and the simulator-extractor extracts a valid decommitment of the $(1 - \tilde{\beta}^*)$ -th inner NM commitment in the SWIAOK system.

That is, with non-negligible probability, the extracted decommitment is for the inner NM commitment that was not used to complete the WIPOK system. Furthermore, since the simulator-extractor simulates the left

⁸For simplicity, we focus on the case that the latter two satisfy *special soundness*. That is, a valid witness can be extracted from any two accepting transcripts if their commit messages are identical and their challenge messages are different.

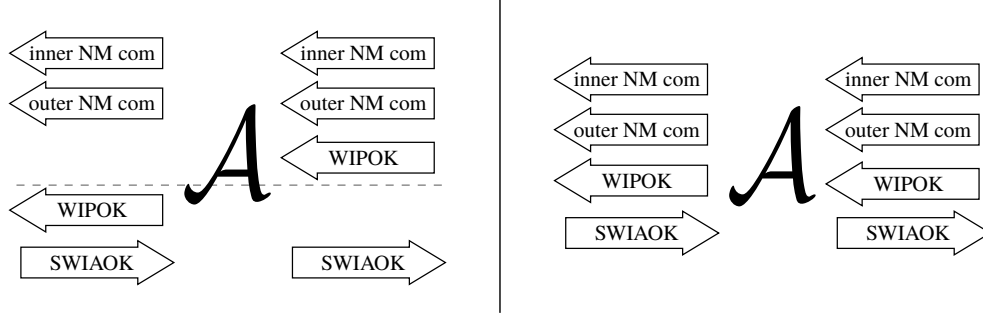


Figure 2: Asynchronous schedule (left) and synchronous schedule (right). In the synchronous schedule, the adversary sends each message in the left interaction immediately after receiving the corresponding message in the right interaction, and vice versa. In the asynchronous schedule, the left and right interactions are synchronized except for the WIPOK part, where the WIPOK part in the left interaction starts after the WIPOK part in the right interaction completes.

interaction by using a decommitment of one of the two inner NM commitments, it follows from an averaging argument that $\exists b^* \in \{0, 1\}$ such that with non-negligible probability, BAD occurs while the simulator-extractor simulates the left interaction using a decommitment of the b^* -th inner NM commitment. Then, consider the following hybrid experiments.

- In the first hybrid, the witness used for SWIAOK in the left interaction is obtained by extracting the committed value of the b^* -th outer NM commitment by brute force (rather than using the extractor of the WIPOK system).

Note that when the simulator-extractor obtains a decommitment of the b^* -th inner NM commitment from the extractor of the WIPOK system, the knowledge soundness of the WIPOK system guarantees that the same decommitment must be committed in the b^* -th outer NM commitment (and thus can be extracted by brute force). Thus, even in this hybrid, BAD occurs with non-negligible probability.

- In the second hybrid, the value committed in the $(1 - \tilde{\beta}^*)$ -th outer NM commitment in the right interaction is switched to an all-0 string.

The (CCA-style) non-malleability of the NM commitment scheme guarantees that even in this hybrid, BAD occurs with non-negligible probability.

Since the decommitment of the $(1 - \tilde{\beta}^*)$ -th inner NM commitment is no longer used in the right interaction in the second hybrid, we can break the non-malleability of the $(1 - \tilde{\beta}^*)$ -th inner NM commitment whenever BAD occurs.

2.2.2 Case 2: NMZK in synchronous message scheduling.

As a more interesting case, consider the setting where the messages in the left and right interactions are scheduled synchronously as shown in Figure 2 (right).

In this case, the above analysis fails when we attempt to show the existence of $\tilde{\beta}^*$ such that BAD occurs with non-negligible probability. To see this, recall that simulating the left interaction involves rewinding the WIPOK system in the left interaction. In the synchronized scheduling, rewinding the left WIPOK system inherently involves rewinding the right WIPOK system. Thus, the existence of $\tilde{\beta}^*$ as above does not follow readily from the witness indistinguishability of the WIPOK system.

To address this issue, we adopt a technique from prior work on 4-round computational NMZK arguments. Concretely, we adapt a technique from Ciampi et al. [COSV17b]. In Ciampi et al., a problem that is closely related to the above is addressed by designing a specific construction of WIPOK. Their WIPOK system is

a careful combination of several low-level primitives (such as a non-interactive commitment scheme and an honest-verifier zero-knowledge system), and the security of their 4-round NMZK argument is analyzed by directly considering reductions to the security of these low-level primitives. We define a WIPOK system that satisfies key properties of their WIPOK system, and integrate it into our construction and analysis in a (somewhat) modular fashion.⁹

Concretely, we consider the following 3-round special-purpose WIPOK system. It is tailored for the statement of the form $x_0 \in \mathbf{L} \vee x_1 \in \mathbf{L}$ (where both $x_0 \in \mathbf{L}$ and $x_1 \in \mathbf{L}$ have a unique witness), and it has the following “delayed-input” syntax that uses x_0 and x_1 only in the last round.

- For every $\beta \in \{0, 1\}$, $(x_\beta, w_\beta) \in \mathbf{R}_\mathbf{L}$, and $x_{1-\beta} \in \{0, 1\}^{|x_\beta|}$,

$$\Pr \left[\text{WIPOK.V}_D(x_0, x_1, \tau) = 1 \mid \begin{array}{l} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^{|x_\beta|}, \beta) \\ \text{wi}_2 \leftarrow \text{WIPOK.V}(1^{|x_\beta|}) \\ \text{wi}_3 \leftarrow \text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, \text{wi}_2) \\ \tau := (\text{wi}_1, \text{wi}_2, \text{wi}_3) \end{array} \right] = 1.$$

Furthermore, its witness indistinguishability is established through a series of hybrid experiments, and some of the hybrid experiments are indistinguishable even against rewinding adversaries. Concretely, there exists a hybrid prover $(\text{WIPOK.}\hat{\text{P}}_1, \text{WIPOK.}\hat{\text{P}}_2)$ satisfying the following.

1. The hybrid prover $(\text{WIPOK.}\hat{\text{P}}_1, \text{WIPOK.}\hat{\text{P}}_2)$ follows the same syntax as the real prover $(\text{WIPOK.P}_1, \text{WIPOK.P}_2)$ except that $\text{WIPOK.}\hat{\text{P}}_2$ needs to have witnesses for both of the sub-statements x_0, x_1 regardless of the value β . That is, it works as $\text{wi}_3 \leftarrow \text{WIPOK.}\hat{\text{P}}_2(\text{st}, x_0, x_1, w_0, w_1, \text{wi}_2)$.
2. The indistinguishability between the hybrid prover $(\text{WIPOK.}\hat{\text{P}}_1, \text{WIPOK.}\hat{\text{P}}_2)$ and the real prover $(\text{WIPOK.P}_1, \text{WIPOK.P}_2)$ can be shown using an auxiliary prover algorithm $\text{WIPOK.}\hat{\text{P}}_{1.5}$ as follows.

$$\begin{aligned} &(\text{WIPOK.}\hat{\text{P}}_1, \text{WIPOK.}\hat{\text{P}}_2) \\ &\equiv ((\text{WIPOK.P}_1, \text{WIPOK.}\hat{\text{P}}_{1.5}), \text{WIPOK.}\hat{\text{P}}_2) \end{aligned} \tag{1}$$

$$\equiv (\text{WIPOK.P}_1, (\text{WIPOK.}\hat{\text{P}}_{1.5}, \text{WIPOK.}\hat{\text{P}}_2)) \tag{2}$$

$$\approx (\text{WIPOK.P}_1, \text{WIPOK.P}_2) \tag{3}$$

That is, (i) the execution of $\text{WIPOK.}\hat{\text{P}}_1$ can be decomposed into a sequential execution of WIPOK.P_1 and $\text{WIPOK.}\hat{\text{P}}_{1.5}$ as shown in (1), (ii) the execution of $\text{WIPOK.}\hat{\text{P}}_{1.5}$ can be deferred until the last round as shown in (2),¹⁰ and (iii) a sequential execution of $\text{WIPOK.}\hat{\text{P}}_{1.5}$ and $\text{WIPOK.}\hat{\text{P}}_2$ is indistinguishable from WIPOK.P_2 as shown in (3).

3. The hybrid prover $(\text{WIPOK.}\hat{\text{P}}_1, \text{WIPOK.}\hat{\text{P}}_2)$ satisfies witness indistinguishability even against rewinding adversaries. That is, any cheating verifier cannot tell whether $\text{WIPOK.}\hat{\text{P}}_1$ runs on input $\beta = 0$ or $\beta = 1$ even given multiple responses from $\text{WIPOK.}\hat{\text{P}}_2$.¹¹

Note that given the hybrid prover as above, the witness indistinguishability indeed follows from a simple hybrid argument: first switch the real prover to the hybrid one, next flip the value of β , and finally switch back the hybrid prover to the real one. (The second step of this hybrid argument holds even against rewinding adversaries.)

⁹Ciampi et al. [COS22] makes a similar attempt at modularization, resulting in a different formalization (called *non-malleable honest-verifier zero-knowledge with respect to commitment*).

¹⁰Specifically, $\text{WIPOK.}\hat{\text{P}}_{1.5}$ only generates auxiliary information to be used in $\text{WIPOK.}\hat{\text{P}}_2$ (without generating anything related to the message to be sent to the verifier in the first round). Thus, any (non-rewinding) verifier cannot detect whether $\text{WIPOK.}\hat{\text{P}}_{1.5}$ is executed in the first round or in the last round.

¹¹This property does not contradict the proof-of-knowledge property since the hybrid prover has witnesses for both sub-statements. In particular, rewinding the hybrid prover results in extracting witnesses for both sub-statements x_0, x_1 , and thus, is useless for determining the value of β .

Using the above special-purpose WIPOK system, we analyze the synchronous case as follows. Recall that in the asynchronous case, we considered a hybrid argument where (i) in the first hybrid, two witnesses are available for the WIPOK system in the right interaction, whereas (ii) in the last hybrid, only a single witness is available. Based on this observation, we modify our construction so that the verifier completes the WIPOK system using the hybrid prover algorithms ($\text{WIPOK}.\hat{P}_1, \text{WIPOK}.\hat{P}_2$). Thus, an honest verifier uses both of the decommitments of the two inner NM commitments for the WIPOK system. On the positive side, this modification enables us to show the existence of $\tilde{\beta}^*$ such that BAD occurs with non-negligible probability. (The key point is that the hybrid prover of the special-purpose WIPOK system is witness indistinguishable even against rewinding adversaries.) On the negative side, this modification requires us to consider an additional hybrid experiment where we switch the hybrid prover ($\text{WIPOK}.\hat{P}_1, \text{WIPOK}.\hat{P}_2$) to the real prover ($\text{WIPOK}.P_1, \text{WIPOK}.P_2$). (Otherwise, the decommitment of the $(1 - \tilde{\beta}^*)$ -th inner NM commitment remains to be used in the right interaction, and therefore we cannot derive a contradiction in the last hybrid.) Given that the hybrid prover and the real prover are not guaranteed to be indistinguishable against rewinding adversaries, it might appear that we have not made any progress. A key point to overcome this difficulty is that we do not require indistinguishability against rewinding adversaries here. To see this, observe that, as evidenced in the analysis of the asynchronous case, we will later replace the rewinding extraction with brute-force extraction in the left interaction. That is, the WIPOK system in the left interaction is not rewound in later hybrids. This observation suggests that we only need to replace the hybrid prover with the real one in the “main thread” and do not need to replace them in the “look-ahead threads” that are generated during the rewinding of the left WIPOK system (cf. Figure 3, top left and right).¹² In other words, we do not need to worry about using the hybrid prover in the look-ahead threads because the look-ahead threads will disappear in later hybrids when we switch the rewinding extraction to brute-force extraction. Concretely, first, we decompose $\text{WIPOK}.\hat{P}_1$ into $(\text{WIPOK}.P_1, \text{WIPOK}.\hat{P}_{1.5})$, and next, use an averaging argument to show that BAD occurs with non-negligible probability even when the main thread and the look-ahead threads use different executions of $\text{WIPOK}.\hat{P}_{1.5}$ (cf. Figure 3, bottom left and right). Then, we rely on the indistinguishability between $(\text{WIPOK}.\hat{P}_{1.5}, \text{WIPOK}.\hat{P}_2)$ and $\text{WIPOK}.P_2$ to show that BAD occurs with non-negligible probability even when $(\text{WIPOK}.\hat{P}_{1.5}, \text{WIPOK}.\hat{P}_2)$ is switched to $\text{WIPOK}.P_2$ in the main thread (cf. Figure 3, top right). The rest of the analysis can proceed as in the asynchronous case since the decommitment of the $(1 - \tilde{\beta}^*)$ -th inner NM commitment is no longer used in the right interaction in the main thread.

Remark 1 (On the use of the special-purpose WIPOK). While the construction of the special-purpose WIPOK is taken from Ciampi et al. [COSV17b] without modifications, we use it in a different way than Ciampi et al. Specifically, our SNMZK construction uses the hybrid prover algorithms ($\text{WIPOK}.\hat{P}_1, \text{WIPOK}.\hat{P}_2$) in its honest execution, whereas their NMZK construction uses the real prover algorithms ($\text{WIPOK}.P_1, \text{WIPOK}.P_2$) in its honest execution. Interestingly, our analysis relies on this particular use of the hybrid prover algorithms, and we do not know whether our construction satisfies SNMZK if the real prover is used in its honest execution. (Thus, our analysis relies on the fact that, in the approach by Feige and Shamir [FS90a], the verifier has two witnesses for the WIPOK system.) To understand the difficulty, suppose the verifier of our construction completes the WIPOK system by using the real prover algorithms. If we follow the analysis above, we encounter a difficulty when we try to show the existence of $\tilde{\beta}^*$ such that BAD occurs with non-negligible probability. To rely on the witness indistinguishability of the hybrid prover algorithms (which holds against rewinding adversaries), we need to consider a hybrid experiment where we switch the real prover algorithms to the hybrid prover algorithms. The problem is that this switch needs to be made in the main thread and all the look-ahead threads. Given that the hybrid prover and the real prover are not guaranteed to be indistinguishable against rewinding adversaries, we do not know how to establish indistinguishability. (This problem is avoided when the hybrid prover algorithms are used in the honest execution. The key is that the hybrid prover algorithms are used in the look-ahead threads from the beginning, and we do not need to consider any switch in the look-ahead threads.) $\diamond$

¹²This technique is first introduced in the context of *security with super-polynomial-time simulation* by Garg et al. [GGJS12]. (Subsequently, it is also used in the context of NMZK by Ciampi et al. [COSV17b].)

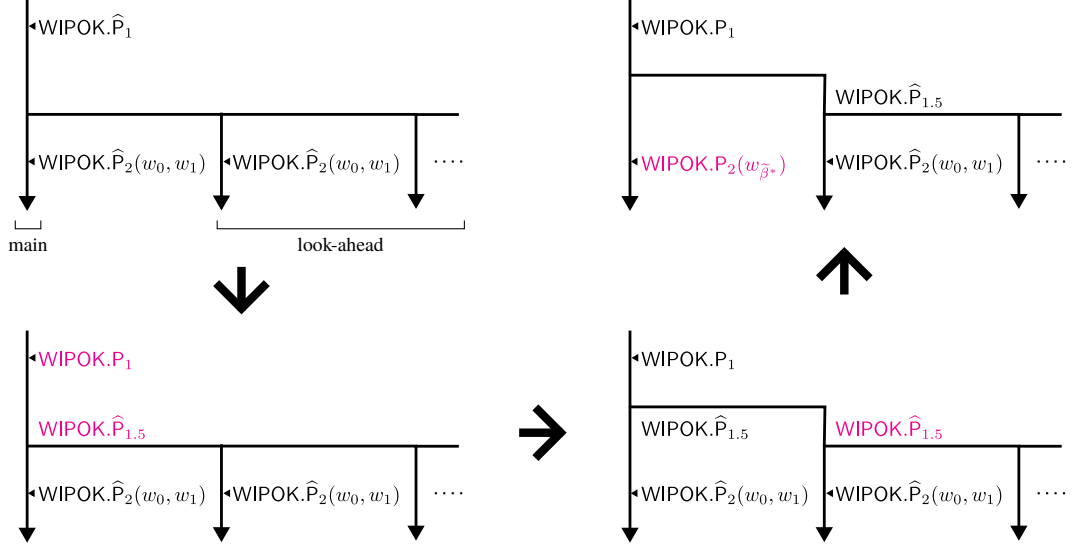


Figure 3: Sequence of hybrids where the hybrid prover of the WIPOK system is removed from the main thread.

2.3 4-round Construction

Finally, we explain how to reduce the round complexity to 4.

The idea is simple: we run all the components of our construction in parallel. As mentioned above, the special-purpose WIPOK system requires only 3 rounds. Since there exist a 3-round NM commitment scheme [GPR16, GR19] and a 4-round delayed-input SWIAOK system [LS91], we can reduce the round complexity to 4 by running all the components in parallel. (There are several minor issues that need to be addressed. For example, the decommitments of the inner NM commitments must be fixed in the first round so that they can be committed to in the outer NM commitments in parallel. Fortunately, the existing 3-round NM commitments [GPR16, GR19] have a commit-challenge-response structure and therefore have the required property.)

The analysis is largely the same as before. Indeed, the synchronous case can be analyzed in the same way as above. For the asynchronous case, when the round complexity is reduced to 4, there are only a limited number of ways to schedule the messages in the left and right interactions, and each of them can be handled without difficulty.

We conclude this technical overview by noting that our construction can be instantiated using any collision-resistant hash function and injective one-way function. The special-purpose WIPOK system can be instantiated using any injective one-way function [COSV17b], the 3-round NM commitment scheme with the required properties can be instantiated using any injective one-way function [GPR16, GR19], and the 4-round delayed-input SWIAOK system can be instantiated using any collision-resistant hash function [LS91, DPP97, HM96].

3 Preliminaries

3.1 Notations and Conventions

We use λ to denote the security parameter. We use ε to denote the empty string. For any $n \in \mathbb{N}$, we use $[n]$ to denote the set $\{1, \dots, n\}$. We use poly to denote an unspecified polynomial, negl to denote an unspecified negligible function, and PPT as an abbreviation of “probabilistic polynomial-time.” For any NP language $\mathbf{L}$, we use $\mathbf{R}_{\mathbf{L}}$ to denote the corresponding witness relation. For any instance $x \in \mathbf{L}$, we use $\mathbf{R}_{\mathbf{L}}(x)$ to denote the set of all witnesses for $x \in \mathbf{L}$. For any pair of probabilistic interactive Turing machines (P, V) and any $x, y, z \in \{0, 1\}^*$, we use $\langle P(y), V(z) \rangle(x)$ to denote the random variable representing the output of V in an interaction between $P(x, y)$ and $V(x, z)$. For a set S , we denote by $s \leftarrow S$ the process of obtaining an element s by a

uniform sampling from S . For any probabilistic algorithm Algo and an input x , we denote by $y \leftarrow \text{Algo}(x)$ the process of obtaining an output y by running $\text{Algo}(x)$ with uniform randomness. When we write $\text{Algo}(x; \text{rand})$, it means that Algo is run with input x and random tape rand . We also use $\text{Algo}(x)$ to denote the range of $\text{Algo}(x; \cdot)$, i.e., $\text{Algo}(x) = \{y \mid \exists r \text{ s.t. } y = \text{Algo}(x; r)\}$.

The reader is assumed to have basic knowledge of cryptography, such as the definitions of statistical/computational indistinguishability, one-way functions, pseudorandom generators, and interactive Turing machines. (For basic definitions, see standard textbooks like [Gol01].) In this paper, every adversarial party is modeled as a non-uniform Turing machine, i.e., takes a non-uniform string (denoted by $z \in \{0, 1\}^*$) as additional input. While we do not explicitly specify the length of non-uniform strings, the length is always assumed to be bounded by a polynomial in λ .¹³

3.2 Interactive Proofs/Arguments

Definition 1 (Interactive proofs/arguments). *Let L be an NP language. A pair of PPT interactive Turing machines (P, V) is called an interactive proof (resp., argument) for L if it satisfies the following.*

Completeness. *For every $(x, w) \in R_L$,*

$$\Pr[\langle P(w), V \rangle(x) = 1] = 1.$$

Soundness. *For every computationally unbounded (resp., PPT) interactive Turing machine $\mathcal{P}$, there exists a negligible function negl such that for every $x \in \{0, 1\}^* \setminus L$ and $z \in \{0, 1\}^*$,*

$$\Pr[\langle \mathcal{P}(z), V \rangle(x) = 1] \leq \text{negl}(|x|) .$$

In the above, the machine P is called the prover, V is called the verifier, x is called the statement, and the sequence of the messages exchanged between P and V is called the transcript. $\diamond$

Definition 2 (public-coin property). *An interactive proof (resp., argument) is called a public-coin interactive proof (resp., argument) if (i) the messages sent by the verifier are random strings that are sampled uniformly and independently in each round and (ii) the verifier deterministically determines its final output based on the statement and the transcript.* $\diamond$

Definition 3 (tag-based interactive proofs/arguments). *An interactive proof (resp., argument) is called a tag-based interactive proof (resp., argument) if the prover and the verifier additionally take a string called a tag as common input.* $\diamond$

In this paper, the notion of tag-based interactive proofs/arguments is used to define non-malleable zero-knowledge (where tags represent provers' identities). Security notions defined for ordinary interactive proofs/arguments are naturally extended for tag-based ones, requiring them to hold for all tags of appropriate length.

3.3 Adaptive-input Interactive Proofs/Arguments

We recall the definition of *adaptive-input interactive proofs/arguments* [LS91, CPS⁺16a, CPS⁺16b], i.e., interactive proofs/arguments such that (i) the statement being proven is used only in the last round and (ii) the security holds even against adversaries selecting the statement adaptively. For concreteness, we focus on 4-round constructions with public verifiability.

¹³That is, for any adversary $\mathcal{A}$, we implicitly fix a polynomial $\text{poly}_{\mathcal{A}}$ such that on security parameter λ , the adversary $\mathcal{A}$ takes as auxiliary input a string z of length at most $\text{poly}_{\mathcal{A}}(\lambda)$.

3.3.1 Basic Security

Definition 4 (adaptive-input interactive proofs/arguments). *Let $\mathbf{L}$ be an NP language. A tuple of PPT Turing machines $(P_1, P_2, V_1, V_2, V_D)$ is called a (4-round) adaptive-input interactive proof (resp, argument) for $\mathbf{L}$ if it satisfies the following.*

Delayed-input completeness. *For every $(x, w) \in R_L$,*

$$\Pr \left[V_D(x, \tau) = 1 \mid \begin{array}{l} (\text{msg}_1, \text{st}_V) \leftarrow V_1(1^{|x|}) \\ (\text{msg}_2, \text{st}_P) \leftarrow P_1(1^{|x|}, \text{msg}_1) \\ \text{msg}_3 \leftarrow V_2(\text{st}_V, \text{msg}_2) \\ \text{msg}_4 \leftarrow P_2(\text{st}_P, x, w, \text{msg}_3) \\ \tau := (\text{msg}_1, \dots, \text{msg}_4) \end{array} \right] = 1.$$

Adaptive-input soundness. *For every pair of computationally unbounded (resp., PPT) Turing machines (P_1, P_2) , there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$,*

$$\Pr \left[\begin{array}{l} x \notin \mathbf{L} \\ \wedge V_D(x, \tau) = 1 \end{array} \mid \begin{array}{l} (\text{msg}_1, \text{st}_V) \leftarrow V_1(1^\lambda) \\ (\text{msg}_2, \text{st}_P) \leftarrow P_1(1^\lambda, \text{msg}_1, z) \\ \text{msg}_3 \leftarrow V_2(\text{st}_V, \text{msg}_2) \\ (x, \text{msg}_4) \leftarrow P_2(\text{st}_P, \text{msg}_3) \\ \tau := (\text{msg}_1, \dots, \text{msg}_4) \end{array} \right] \leq \text{negl}(\lambda).$$

In the above, the messages msg_2 , msg_3 , and msg_4 are called the commit, challenge, and response messages. $\diamond$

Remark 2. In the case of public-coin adaptive-input interactive proof/argument, the syntax of the verifier is simplified by removing st_V from the output of V_1 and only giving $1^{|x|}$ to V_2 as input. $\diamond$

3.3.2 Additional Security

First, we provide the definition of *adaptive-input computational special soundness*. The following definition is based on the definition of *computational special soundness* by Pass [Pas11] and the definition of *adaptive-input special soundness* by Ciampi et al. [CPS⁺16b].

Definition 5 (adaptive-input computational special soundness). *Let $\Pi = (P_1, P_2, V_1, V_2, V_D)$ be a public-coin adaptive-input interactive argument for an NP language $\mathbf{L}$. Then, Π is said to satisfy adaptive-input computational special soundness if there exists a deterministic polynomial-time Turing machine Ext (called the extractor) such that for every pair of PPT Turing machines (P_1, P_2) , there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$,*

$$\Pr \left[\begin{array}{l} \forall b \in \{0, 1\} : V_D(x_b, \tau_b) = 1 \\ \wedge (x_0, w_0) \notin R_L \end{array} \mid \begin{array}{l} \text{msg}_1 \leftarrow V_1(1^\lambda) \\ (\text{msg}_2, \text{st}_P) \leftarrow P_1(1^\lambda, \text{msg}_1, z) \\ \forall b \in \{0, 1\} : \text{msg}_{b,3} \leftarrow V_2(1^\lambda) \\ (x_0, \text{msg}_{0,4}, x_1, \text{msg}_{1,4}) \leftarrow P_2(\text{st}_P, \text{msg}_{0,3}, \text{msg}_{1,3}) \\ \forall b \in \{0, 1\} : \tau_b := (\text{msg}_1, \text{msg}_2, \text{msg}_{b,3}, \text{msg}_{b,4}) \\ w_0 := \text{Ext}(x_0, \tau_0, x_1, \tau_1) \end{array} \right] \leq \text{negl}(\lambda).$$

$\diamond$

Next, we recall the definition of *adaptive-input statistical witness indistinguishability*.

Definition 6 (adaptive-input statistical WI). *Let $\Pi = (P_1, P_2, V_1, V_2, V_D)$ be an adaptive-input interactive argument for an NP language $\mathbf{L}$. Then, Π is called adaptive-input statistical witness-indistinguishable (WI) if for every triple of computationally unbounded Turing machines (V_1, V_2, V_3) , there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$, the random variable out satisfies $\Pr [\text{out} = 1] \leq 1/2 + \text{negl}(\lambda)$ in the following experiment.*

1. Compute $(\text{msg}_1, \text{st}_{V,1}) \leftarrow \mathcal{V}_1(1^\lambda, z)$, $(\text{msg}_2, \text{st}_P) \leftarrow P_1(1^\lambda, \text{msg}_1)$, and $(\text{msg}_3, x, w_0, w_1, \text{st}_{V,2}) \leftarrow \mathcal{V}_2(\text{st}_{V,1}, \text{msg}_2)$. If $|x| \neq \lambda \vee \exists b \in \{0, 1\} \text{ s.t. } (x, w_b) \notin \mathbf{R}_L$, abort.
2. Sample a random bit $\beta \in \{0, 1\}$ and compute $\text{msg}_4 \leftarrow P_2(\text{st}_P, x, w_\beta, \text{msg}_3)$.
3. Compute $\tilde{\beta} \leftarrow \mathcal{V}_3(\text{st}_{V,2}, \text{msg}_4)$ and let $\text{out} = 1$ if and only if $\tilde{\beta} = \beta$. $\diamond$

Finally, we provide an instantiation that satisfies the above two additional security notions.

Theorem 1. *Assume the existence of a collision-resistant hash function family. Then, there exists a 4-round public-coin adaptive-input interactive argument that satisfies both adaptive-input computational special soundness and adaptive-input statistical witness indistinguishability.*

Proof sketch. It suffices to instantiate the delayed-input argument of knowledge of Lapidot and Shamir [LS91] with a 2-round statistically hiding commitment scheme (which can be obtained from any collision-resistant hash function [DPP97, HM96]) and use a trick by Ciampi et al. [COSV17b, Appendix C.1] to obtain delayed-input computational special soundness. $\square$

3.4 Statistical Non-malleable Zero-knowledge

We recall the definition of *non-malleable zero-knowledge* [DDN00]. Following recent work [GRRV21, COSV17c, COSV17a, BCO⁺24], we use the definition based on the notion of *simulation extractability* [PR08b] (which is known to imply the original definition of non-malleable zero-knowledge [DDN00]). Below, we provide the statistical zero-knowledge version of the definition in the *one-many* setting [PR08a] with a weak form of *adaptive input selection* [LP11a]. (See Remark 3 below for details about adaptive input selection.) The text below is adapted from [LP11a, Kiy20].

Let $\langle P, V \rangle$ be a tag-based, adaptive-input interactive argument for an NP language L . For a man-in-the-middle adversary $\mathcal{A}$, let us consider a probabilistic experiment in which $\mathcal{A}$ participates in the following single left and multiple right interactions.

- In the left interaction, $\mathcal{A}$ interacts with an honest prover of $\langle P, V \rangle$ and verifies the validity of a statement x using a tag tag .
- In the right interactions, $\mathcal{A}$ interacts with honest verifiers of $\langle P, V \rangle$ and proves the validity of statements $\tilde{x}_1, \dots, \tilde{x}_m$ using tags $\tilde{\text{tag}}_1, \dots, \tilde{\text{tag}}_m$.

More concretely, the experiment proceeds as follows. Prior to the interactions, all parties in the experiment receive as common input the security parameter λ in unary, and $\mathcal{A}$ also receives as auxiliary input $z \in \{0, 1\}^*$. Then, $\mathcal{A}$ starts interacting with the prover and the verifiers in the left and right interactions while fully controlling the message schedule. (In particular, $\mathcal{A}$ can run the left and right interactions concurrently.) In the left (resp., each right) interaction, the tag $\text{tag} \in \{0, 1\}^\lambda$ (resp., $\tilde{\text{tag}}_i \in \{0, 1\}^\lambda$) is selected by $\mathcal{A}$ at the beginning of the interaction, and the statement $x \in \{0, 1\}^\lambda$ (resp., $\tilde{x}_i \in \{0, 1\}^\lambda$) is selected by $\mathcal{A}$ at the time of sending the last verifier (resp., prover) message to the honest party. The witness used by the honest prover in the left interaction is chosen by a *witness-selecting machine* M . Specifically, M is a PPT Turing machine such that when $\mathcal{A}$ selects the statement x in the left interaction, M takes as input the statement x , the current view of $\mathcal{A}$, and an auxiliary input z' , and then picks a witness w as the private input to the honest prover. Let $\text{view}_{\mathcal{A}, M}(\lambda, z, z')$ be the random variable representing the joint view of $\mathcal{A}$ and the honest verifiers in the above experiment. Let us say that the pair (z, z') of auxiliary inputs is *good* if in the above experiment, (i) $\mathcal{A}$ always selects a true statement of length λ in the left interaction and (ii) M always picks a valid witness for the validity of the statement selected by $\mathcal{A}$. Let $Z_{\mathcal{A}, M}(\lambda)$ be the set of good auxiliary-input pairs. Then, roughly speaking, $\langle P, V \rangle$ is *one-many statistical non-malleable zero-knowledge (SNMZK) with weak adaptive-input selection* if for every PPT adversary $\mathcal{A}$ and witness-selecting machine M , there exists a PPT machine called the *simulator-extractor* such that for every λ and every good (z, z') , the simulator-extractor can statistically simulate the view of $\mathcal{A}$ in the above experiment while

extracting witnesses for the statements proven by $\mathcal{A}$ in the accepted right interactions. The formal definition is given below.

Definition 7. A tag-based adaptive-input interactive argument $\langle P, V \rangle$ for an NP language L is said to be one-many statistical non-malleable zero-knowledge with weak adaptive input selection if for every polynomial $m(\cdot)$ and every PPT man-in-the-middle adversary $\mathcal{A}$ that participates in at most $m = m(\lambda)$ right interactions, there exists an expected polynomial-time Turing machine $\mathcal{SE}$ (called the simulator-extractor) such that for every PPT witness-selecting machine M , the following hold.

1. **Statistical simulation.** Let $\text{sim-view}(\lambda, z)$ be a random variable representing the first output of $\mathcal{SE}(1^\lambda, z)$. Then, for every sequence $\{(z_\lambda, z'_\lambda)\}_{\lambda \in \mathbb{N}}$ such that $(z_\lambda, z'_\lambda) \in Z_{\mathcal{A}, M}(\lambda)$, the ensembles $\{\text{view}_{\mathcal{A}, M}(\lambda, z_\lambda, z'_\lambda)\}_{\lambda \in \mathbb{N}}$ and $\{\text{sim-view}(\lambda, z_\lambda)\}_{\lambda \in \mathbb{N}}$ are statistically indistinguishable.
2. **Witness extraction.** For every $\lambda \in \mathbb{N}$ and $(z, z') \in Z_{\mathcal{A}, M}(\lambda)$, the following holds. Let $(\text{view}, \{\tilde{w}_i\}_{i \in [m]})$ denote the output of $\mathcal{SE}(\lambda, z)$. Let $\tilde{x}_1, \dots, \tilde{x}_m$ be the statements of the right interactions in view , and let tag and $\tilde{\text{tag}}_1, \dots, \tilde{\text{tag}}_m$ be the tags of the left and the right interactions in view . Then, except with negligible probability, it holds $(\tilde{x}_i, \tilde{w}_i) \in \mathbf{R}_L$ for every $i \in [m]$ such that the i -th right interaction is accepting and $\tilde{\text{tag}}_i \neq \text{tag}$. $\diamond$

Remark 3. The above formalization of adaptive input selection is weaker than the original one [LP11a] because the original formalization considers the case where the witness-selecting machine is computationally unbounded and takes the views of all the parties (including the honest ones) as input. In other words, we focus on the case where a witness for the left statement x can be computed efficiently by using a “super witness” z' . (See [LP11a, Section 1] for more discussion.) $\diamond$

3.5 Synchronous Non-malleable Commitments

We recall the definition of *non-malleable commitment schemes* [DDN00]. The following definition is based on the definition by Lin et al. [LPV08]. For concreteness, we focus on 3-round constructions with public verifiability¹⁴ in the synchronous-message setting.

Definition 8. A tuple of PPT Turing machines $(\text{NMC.S}_1, \text{NMC.S}_2, \text{NMC.R}, \text{NMC.R}_D)$ is called a 3-round synchronous non-malleable commitment scheme if it satisfies the following.

Correctness. For every $\lambda \in \mathbb{N}$, $\text{tag} \in \{0, 1\}^\lambda$, and $m \in \{0, 1\}^\lambda$,

$$\Pr \left[\text{NMC.R}_D(\tau, \text{dec}) = m \mid \begin{array}{l} (\text{nm}_1, \text{dec}, \text{st}) \leftarrow \text{NMC.S}_1(1^\lambda, \text{tag}, m) \\ \text{nm}_2 \leftarrow \text{NMC.R}(1^\lambda, \text{tag}, \text{nm}_1) \\ \text{nm}_3 \leftarrow \text{NMC.S}_2(\text{st}, \text{nm}_2) \\ \tau := (\text{nm}_1, \text{nm}_2, \text{nm}_3) \end{array} \right] = 1.$$

Statistical binding. For every pair of computationally unbounded Turing machines $(\mathcal{S}_1, \mathcal{S}_2)$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$,

$$\Pr \left[\begin{array}{l} m \neq \perp \wedge m' \neq \perp \\ \wedge \text{NMC.R}_D(\tau, \text{dec}) = m \\ \wedge \text{NMC.R}_D(\tau, \text{dec}') = m' \end{array} \mid \begin{array}{l} (\text{tag}, \text{nm}_1, \text{st}) \leftarrow \mathcal{S}_1(1^\lambda, z) \\ \text{nm}_2 \leftarrow \text{NMC.R}(1^\lambda, \text{tag}, \text{nm}_1) \\ (\text{nm}_3, m, \text{dec}, m', \text{dec}') \leftarrow \mathcal{S}_2(\text{st}, \text{nm}_2) \\ \tau := (\text{nm}_1, \text{nm}_2, \text{nm}_3) \end{array} \right] \leq \text{negl}(\lambda).$$

Synchronous non-malleability. For any tuple of PPT Turing machines $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_D)$, there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$, the random variable out satisfies $\Pr[\text{out} = 1] \leq 1/2 + \text{negl}(\lambda)$ in the following experiment.

¹⁴That is, decommitment verification does not require the receiver’s internal state.

Initialization. Compute $(m, \text{tag}, \text{st}_{\mathcal{A},0}) \leftarrow \mathcal{A}_0(1^\lambda, z)$, where $m \in \{0, 1\}^\lambda$, $\text{tag} \in \{0, 1\}^\lambda$.¹⁵ Then, let m^* be defined as follows: sample a random bit $b \in \{0, 1\}$; if $b = 0$, let $m^* := m$; otherwise, let $m^* := m'$ for a random string $m' \in \{0, 1\}^\lambda$.

Round 1. Compute $(\text{nm}_1, \text{dec}, \text{st}_S) \leftarrow \text{NMC.S}_1(1^\lambda, \text{tag}, m^*)$ and $(\widetilde{\text{tag}}, \widetilde{\text{nm}}_1, \text{st}_{\mathcal{A},1}) \leftarrow \mathcal{A}_1(\text{st}_{\mathcal{A},0}, \text{nm}_1)$.

Round 2. Compute $\widetilde{\text{nm}}_2 \leftarrow \text{NMC.R}(1^\lambda, \widetilde{\text{tag}}, \widetilde{\text{nm}}_1)$ and $(\text{nm}_2, \text{st}_{\mathcal{A},2}) \leftarrow \mathcal{A}_2(\text{st}_{\mathcal{A},1}, \widetilde{\text{nm}}_2)$.

Round 3. Compute $\text{nm}_3 \leftarrow \text{NMC.S}_2(\text{st}_S, \text{nm}_2)$ and $(\text{st}_{\mathcal{A},3}, \widetilde{\text{nm}}_3) \leftarrow \mathcal{A}_3(\text{st}_{\mathcal{A},2}, \text{nm}_3)$.

Final output. Let $\widetilde{m}$ be the committed value of $\widetilde{\tau} := (\widetilde{\text{nm}}_1, \widetilde{\text{nm}}_2, \widetilde{\text{nm}}_3)$ defined as follows: let $\widetilde{M} := \{\widetilde{m} \mid \exists \widetilde{\text{dec}} \text{ s.t. } \text{NMC.R}_D(\widetilde{\tau}, \widetilde{\text{dec}}) = \widetilde{m}\}$; if $\text{tag} = \widetilde{\text{tag}}$ or $|\widetilde{M}| \neq 1$, let $\widetilde{m} := \perp$; otherwise, let $\widetilde{m}$ be the unique value such that $\widetilde{m} \in \widetilde{M}$. Then, compute $b' \leftarrow \mathcal{A}_D(\text{st}_{\mathcal{A},3}, \widetilde{m})$ and let $\text{out} := 1$ if and only if $b' = b$. $\diamond$

We note that the above definition uses somewhat unconventional syntax where the decommitment verification algorithm NMC.R_D outputs either the committed value (when given a valid decommitment) or $\perp$ (when given an invalid decommitment). We also note that synchronous non-malleability trivially implies the following hiding property.

Computational hiding. For any tuple of PPT Turing machines $\mathcal{R} = (\mathcal{R}_0, \mathcal{R}_1, \mathcal{R}_D)$, there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$, the random variable out satisfies $\Pr[\text{out} = 1] \leq 1/2 + \text{negl}(\lambda)$ in the following experiment.

Initialization. Compute $(m, \text{tag}, \text{st}_{\mathcal{R},0}) \leftarrow \mathcal{R}_0(1^\lambda, z)$, where $m \in \{0, 1\}^\lambda$, $\text{tag} \in \{0, 1\}^\lambda$. Then, let m^* be defined as follows: sample a random bit $b \in \{0, 1\}$; if $b = 0$, let $m^* := m$; otherwise, let $m^* := m'$ for a random string $m' \in \{0, 1\}^\lambda$.

Rounds 1, 2, 3. Compute $(\text{nm}_1, \text{dec}, \text{st}_S) \leftarrow \text{NMC.S}_1(1^\lambda, \text{tag}, m^*)$, $(\text{nm}_2, \text{st}_{\mathcal{R},1}) \leftarrow \mathcal{R}_1(\text{st}_{\mathcal{R},0}, \text{nm}_1)$, and $\text{nm}_3 \leftarrow \text{NMC.S}_2(\text{st}_S, \text{nm}_2)$.

Final output. Compute $b' \leftarrow \mathcal{R}_D(\text{st}_{\mathcal{R},1}, \text{nm}_3)$ and let $\text{out} := 1$ if and only if $b' = b$.

As mentioned in Section 2.3, we require a non-malleable commitment scheme that satisfies an additional property. The necessary property is summarized in the following theorem. At a high level, we require that any Round-1 message uniquely determines a “decommitment candidate” that can serve as a valid decommitment when the commit phase is completed honestly.¹⁶ In particular, we require that (i) any valid decommitment is also a valid decommitment candidate and (ii) any valid decommitment candidate is also a valid decommitment when an honest committer completes the commit phase without aborting.

Theorem 2. Assume the existence of injective one-way functions. Then, there exists a 3-round synchronous non-malleable commitment scheme satisfying the following property.

Unique decommitment candidate. There exists a deterministic Turing machine $\text{NMC.R}_{D,\text{weak}}$ satisfying the following.

Uniqueness. For every $\text{nm}_1 \in \{0, 1\}^*$, $|\{\text{dec} \mid \text{NMC.R}_{D,\text{weak}}(\text{nm}_1, \text{dec}) = 1\}| \leq 1$.

Correctness. For every $\tau = (\text{nm}_1, \text{nm}_2, \text{nm}_3) \in (\{0, 1\}^*)^3$ and dec such that $\text{NMC.R}_D(\tau, \text{dec}) \neq \perp$, it holds $\text{NMC.R}_{D,\text{weak}}(\text{nm}_1, \text{dec}) = 1$.

Soundness. For every computationally unbounded Turing machine $\mathcal{A}$ and every $\lambda \in \mathbb{N}$, $\text{tag} \in \{0, 1\}^\lambda$, $m \in \{0, 1\}^\lambda$, and $z \in \{0, 1\}^*$,

$$\Pr \left[\begin{array}{l} \exists \text{dec}' \text{ s.t.} \\ \text{NMC.R}_{D,\text{weak}}(\text{nm}_1, \text{dec}') = 1 \\ \wedge \text{nm}_3 \neq \perp \\ \wedge \text{NMC.R}_D(\tau, \text{dec}') \neq m \end{array} \mid \begin{array}{l} (\text{nm}_1, \text{dec}, \text{st}) \leftarrow \text{NMC.S}_1(1^\lambda, \text{tag}, m) \\ \text{nm}_2 \leftarrow \mathcal{A}(1^\lambda, \text{tag}, \text{nm}_1, z) \\ \text{nm}_3 \leftarrow \text{NMC.S}_2(\text{st}, \text{nm}_2) \\ \tau := (\text{nm}_1, \text{nm}_2, \text{nm}_3) \end{array} \right] = 0.$$

¹⁵If $m \notin \{0, 1\}^\lambda$ or $\text{tag} \notin \{0, 1\}^\lambda$, then m and tag are redefined to be arbitrary default values of length λ .

¹⁶We do not require that the Round-1 message uniquely determines the committed value.

Furthermore, when a commitment is made on security parameter λ and message length λ , the length of the corresponding decommitment is λ .

Proof. The first part follows from [GPR15, Claim 5 and Theorem 1], where the unique decommitment candidate property can be verified by inspecting the protocol description in [GPR15, Figure 2]. Specifically, it suffices to see that (i) the commit phase has a commit-challenge-response structure such that the committer gives a non-interactive commitment in the first round, the receiver responds with a public-coin challenge, and the committer replies with a response message in the third round, and (ii) the decommitment consists of the decommitment of the non-interactive commitment. (The receiver checks the validity of the decommitment w.r.t. the non-interactive commitment and then checks whether the decommitted value is consistent with the transcript of the commit phase.) Let $\text{NMC.R}_{\text{D,weak}}$ be an algorithm that checks whether the given decommitment is a valid decommitment of the non-interactive commitment, and suppose that the underlying non-interactive commitment scheme is instantiated with the perfectly binding non-interactive bit-commitment scheme based on the hard-core predicate of an injective one-way function [Blu82]. Then, the uniqueness holds because the non-interactive commitment has a unique decommitment. The correctness and soundness follow from the construction.

The furthermore part can also be verified by inspection. Specifically, it suffices to recall that the decommitment of the underlying non-interactive commitment consists of the committer randomness, whose length can be shortened to λ by using a pseudorandom generator (without sacrificing the computational hiding property of the underlying non-interactive commitment).¹⁷ The unique decommitment candidate property is preserved as long as the pseudorandom generator is injective, and an injective pseudorandom generator can be constructed from any injective one-way function. $\square$

4 Tool: Special-purpose WIPOK System

We present the main tool that we use in our 4-round SNMZK argument system. As described in Section 2, it is a 3-round public-coin proof of knowledge for statements of the form $x_0 \in \mathbf{L} \vee x_1 \in \mathbf{L}$, and we require it to satisfy a certain rewinding security notion. We start by defining the syntax of such 3-round proof of knowledge systems and their basic security.

Definition 9. Let $\mathbf{L}$ be an NP language. A tuple of PPT Turing machines $(\text{WIPOK.P}_1, \text{WIPOK.P}_2, \text{WIPOK.V}, \text{WIPOK.V}_\text{D})$ is called a 3-round adaptive-input OR-indistinguishable special-sound proof for $\mathbf{L}$ if it satisfies the following.

Delayed-input correctness. For every $\beta \in \{0, 1\}$, $(x_\beta, w_\beta) \in \mathbf{R}_\mathbf{L}$, and $x_{1-\beta} \in \{0, 1\}^{|x_\beta|}$,

$$\Pr \left[\text{WIPOK.V}_\text{D}(x_0, x_1, \tau) = 1 \mid \begin{array}{l} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^{|x_\beta|}, \beta) \\ \text{wi}_2 \leftarrow \text{WIPOK.V}(1^{|x_\beta|}) \\ \text{wi}_3 \leftarrow \text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, \text{wi}_2) \\ \tau := (\text{wi}_1, \text{wi}_2, \text{wi}_3) \end{array} \right] = 1.$$

Delayed-input special soundness. There exists a deterministic polynomial-time Turing machine Ext (called the extractor) and a polynomial-time computable function $\text{EQ} : \{0, 1\}^* \times \{0, 1\}^* \mapsto \{0, 1\}$ such that the following hold.

1. For every $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{EQ}(\text{wi}_{0,2}, \text{wi}_{1,2}) = 1 \mid \begin{array}{l} \text{wi}_{0,2} \leftarrow \text{WIPOK.V}(1^\lambda) \\ \text{wi}_{1,2} \leftarrow \text{WIPOK.V}(1^\lambda) \end{array} \right] \leq \frac{1}{2^\lambda}.$$

¹⁷Since the underlying non-interactive commitment scheme is a bit commitment scheme, the committed value can be efficiently recovered from the commitment and the committer randomness.

2. Fix any $x_b = (x_{b,0}, x_{b,1}) \in (\{0,1\}^*)^2$ and $\tau_b := (w_{b,1}, w_{b,2}, w_{b,3}) \in (\{0,1\}^*)^3$ for each $b \in \{0,1\}$. Suppose it holds $|x_0| = |x_1|$, $w_{0,1} = w_{1,1}$, $\text{EQ}(w_{0,2}, w_{1,2}) = 0$, and $V_D(x_{b,0}, x_{b,1}, \tau_b) = 1$ for every $b \in \{0,1\}$. Then, $\text{Ext}(x_0, \tau_0, x_1, \tau_1)$ outputs a value w such that $\exists \beta \in \{0,1\} : (x_{\beta,0}, w) \in \mathbf{R}_L$.

Adaptive-input OR indistinguishability. For every pair of PPT Turing machines $(\mathcal{V}_1, \mathcal{V}_2)$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0,1\}^*$, the random variable out satisfies $\Pr[\text{out} = 1] \leq 1/2 + \text{negl}(\lambda)$ in the following experiment.

1. Sample a random bit $\beta \in \{0,1\}$ and compute $(w_1, \text{st}_P) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta)$.
2. Compute $(w_2, \{x_b, w_b\}_{b \in \{0,1\}}, \text{st}_V) \leftarrow \mathcal{V}_1(1^\lambda, w_1, z)$. If $\exists b \in \{0,1\}$ s.t. $(x_b, w_b) \notin \mathbf{R}_L \vee |x_b| \neq \lambda$, abort.
3. Compute $w_3 \leftarrow \text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, w_2)$.
4. Compute $\tilde{\beta} \leftarrow \mathcal{V}_2(\text{st}_V, w_3)$ and let $\text{out} = 1$ if and only if $\tilde{\beta} = \beta$.

Remark 4. The adaptive-input OR indistinguishability is weaker than the standard witness indistinguishability since it only requires indistinguishability between a proof generated with a witness for $x_0 \in \mathbf{L}$ and a proof generated with a witness for $x_1 \in \mathbf{L}$. $\diamond$

Next, we summarize the required rewinding security notion in the following theorem.

Theorem 3. Let $\mathbf{L}$ be an NP language, and assume the existence of injective one-way functions. Then, there exists a 3-round adaptive-input OR-indistinguishable special-sound proof, $\text{WIPOK} = (\text{WIPOK.P}_1, \text{WIPOK.P}_2, \text{WIPOK.V}, \text{WIPOK.V}_D)$, for $\mathbf{L}$. Furthermore, there exists a pair of PPT Turing machines $(\text{WIPOK.P}_1, \text{WIPOK.P}_2)$ and an additional PPT Turing machine $\text{WIPOK.P}_{1.5}$ satisfying the following.

Delayed-input correctness of hybrid prover. For every $\beta \in \{0,1\}$, $(x_0, w_0) \in \mathbf{R}_L$, and $(x_1, w_1) \in \mathbf{R}_L$ such that $|x_0| = |x_1|$,

$$\Pr \left[\text{WIPOK.V}_D(x_0, x_1, \tau) = 1 \mid \begin{array}{l} (w_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.P}_1(1^{|x_\beta|}, \beta) \\ w_2 \leftarrow \text{WIPOK.V}(1^{|x_\beta|}) \\ w_3 \leftarrow \text{WIPOK.P}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, w_2) \\ \tau := (w_1, w_2, w_3) \end{array} \right] = 1.$$

Indistinguishability between real and hybrid provers.

1. For every $\lambda \in \mathbb{N}$ and $\beta \in \{0,1\}$, the following two distributions are identically distributed.

$$\left\{ (w_1, \text{st}, \text{aux}) \mid \begin{array}{l} (w_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ \text{aux} \leftarrow \text{WIPOK.P}_{1.5}(1^\lambda, \beta) \end{array} \right\} \\ \left\{ (w_1, \text{st}, \text{aux}) \mid (w_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \right\}.$$

2. For every PPT Turing machine $\mathcal{D}$, there exists a negligible function negl such that for every $(x_0, w_0) \in \mathbf{R}_L$, $(x_1, w_1) \in \mathbf{R}_L$, $\beta \in \{0,1\}$, $(w_1, \text{st}) \in \text{WIPOK.P}_1(1^{|x_\beta|}, \beta)$, $w_2 \in \{0,1\}^*$, and $z \in \{0,1\}^*$ such that $|x_0| = |x_1|$,

$$\left| \Pr[\mathcal{D}(\text{params}, w_3) = 1 \mid w_3 \leftarrow \text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, w_2)] - \Pr \left[\mathcal{D}(\text{params}, w_3) = 1 \mid \begin{array}{l} \text{aux} \leftarrow \text{WIPOK.P}_{1.5}(1^{|x_\beta|}, \beta) \\ w_3 \leftarrow \text{WIPOK.P}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, w_2) \end{array} \right] \right| \leq \text{negl}(|x_\beta|),$$

where $\text{params} := (x_0, x_1, w_0, w_1, \beta, w_1, w_2, \text{st}, z)$.¹⁸

¹⁸Note that aux is not given to $\mathcal{D}$.

Adaptive-input (rewinding) OR-indistinguishability of hybrid prover. For every PPT oracle Turing machine $\mathcal{V}$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0, 1\}^*$,

$$\Pr \left[\beta = \tilde{\beta} \mid \begin{array}{l} \beta \leftarrow \{0, 1\} \\ (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ \tilde{\beta} \leftarrow \mathcal{V}^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(1^\lambda, \text{wi}_1, z) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where the oracle $\mathcal{O}$ is defined as follows.

- $\mathcal{O}(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$: If $|x_b| = \lambda$ and $(x_b, w_b) \in \mathbf{R}_L$ for every $b \in \{0, 1\}$, return $\text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$. Otherwise, return $\perp$.

As stated in Section 2, Theorem 3 can be proved by using the WIPOK system given by Ciampi et al. [COSV17b]. (For concreteness, we provide a proof in Appendix A.) In the following, we give a lemma that we use in the analysis of our 4-round SNMZK argument. Intuitively, the lemma formalizes the argument sketched in Figure 3 in Section 2.2.

Lemma 1. Fix any computationally unbounded Turing machine $\mathcal{A}_1$ and pair of PPT Turing machines $(\mathcal{A}_2, \hat{\mathcal{A}}_2)$ such that for every $\lambda \in \mathbb{N}$, $\beta \in \{0, 1\}$, and $z \in \{0, 1\}$, $\mathcal{A}_1$ is well-behaving in the following sense.

$$\Pr \left[\begin{array}{l} |\text{st}_{\mathcal{A}}| \leq \text{poly}(\lambda) \\ \wedge \forall b \in \{0, 1\} : |x_b| = \lambda \wedge (x_b, w_b) \in \mathbf{R}_L \end{array} \mid \begin{array}{l} (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \end{array} \right] = 1.$$

Also, let $\mathcal{O}$ be the following oracle.

- $\mathcal{O}(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$: If $|x_b| = \lambda$ and $(x_b, w_b) \in \mathbf{R}_L$ for every $b \in \{0, 1\}$, return $\text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$. Otherwise, return $\perp$.

Then, there exists a negligible function negl such that for any polynomial p , there exists a polynomial q such that for every $\lambda \in \mathbb{N}$, $\beta \in \{0, 1\}$, and $z \in \{0, 1\}^*$, if it holds

$$\Pr \left[\begin{array}{l} \mathcal{A}_2(\text{st}_{\mathcal{A}}, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}}) = 1 \end{array} \mid \begin{array}{l} (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \geq \frac{1}{p(\lambda)}, \quad (4)$$

then it also holds

$$\Pr \left[\begin{array}{l} \mathcal{A}_2(\text{st}_{\mathcal{A}}, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}}) = 1 \end{array} \mid \begin{array}{l} (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\text{P}_2(\text{st}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \geq \frac{1}{q(\lambda)} - \text{negl}(\lambda). \quad (5)$$

We emphasize that the oracle $\mathcal{O}$ remains to use hybrid prover algorithm $\text{WIPOK}.\hat{\text{P}}_2$ in (5). (In the terminology used in Section 2.2, adversary $\mathcal{A}_2$ represents the “main thread” and $\hat{\mathcal{A}}_2$ represents the “look-ahead threads.”)

Proof of Lemma 1. Fix any p , λ , β , and $z \in \{0, 1\}^*$ such that we have (4). Let $\varepsilon := 1/p(\lambda)$. From (4) and the indistinguishability between the real and hybrid provers, we have

$$\Pr \left[\begin{array}{l} \mathcal{A}_2(\text{st}_{\mathcal{A}}, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}}) = 1 \end{array} \mid \begin{array}{l} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK}.\text{P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \geq \varepsilon. \quad (6)$$

Thus, it follows from an averaging argument that with probability at least $\varepsilon/2$ over the choice of $(\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta)$ and $(x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z)$, we have

$$\Pr \left[\begin{array}{c} \mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_A) = 1 \end{array} \middle| \begin{array}{c} \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \geq \frac{\varepsilon}{2}. \quad (7)$$

Let G be the set that consists of every $(\text{wi}_1, \text{st}, x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A)$ on which we have (7). For every $(\text{wi}_1, \text{st}, x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \in G$, Equation (7) trivially implies the following two lower bounds.

$$\Pr \left[\mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \middle| \begin{array}{c} \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \geq \frac{\varepsilon}{2}. \quad (8)$$

$$\Pr \left[\hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_A) = 1 \middle| \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \right] \geq \frac{\varepsilon}{2}. \quad (9)$$

Thus, when we add one more execution of $\text{WIPOK}.\hat{\text{P}}_{1.5}$ to the left-side hand of (6) so that the execution of $\hat{\mathcal{A}}_2$ becomes independent of aux , we have

$$\begin{aligned} & \Pr \left[\begin{array}{c} \mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}', \cdot)}(\text{st}_A) = 1 \end{array} \middle| \begin{array}{c} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \\ \text{aux}' \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \end{array} \right] \\ & \geq \Pr \left[(\text{wi}_1, \text{st}, x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \in G \middle| \begin{array}{c} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \end{array} \right] \\ & \quad \times \Pr \left[\mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \middle| \begin{array}{c} \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] \\ & \quad \times \Pr \left[\hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}', \cdot)}(\text{st}_A) = 1 \middle| \text{aux}' \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \right] \\ & \geq \frac{\varepsilon}{2} \cdot \frac{\varepsilon}{2} \cdot \frac{\varepsilon}{2} = \frac{\varepsilon^3}{8}. \end{aligned} \quad (10)$$

We now analyze the left-side hand of (5). Using the indistinguishability between the real and hybrid provers, we obtain

$$\begin{aligned} & \Pr \left[\begin{array}{c} \mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}', \cdot)}(\text{st}_A) = 1 \end{array} \middle| \begin{array}{c} (\text{wi}_1, \text{st}, \text{aux}') \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{wi}_3 \leftarrow \text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, \text{wi}_2) \end{array} \right] \\ & \geq \Pr \left[\begin{array}{c} \mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}', \cdot)}(\text{st}_A) = 1 \end{array} \middle| \begin{array}{c} (\text{wi}_1, \text{st}, \text{aux}') \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \end{array} \right] - \text{negl}(\lambda) \\ & = \Pr \left[\begin{array}{c} \mathcal{A}_2(\text{st}_A, \text{wi}_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}', \cdot)}(\text{st}_A) = 1 \end{array} \middle| \begin{array}{c} (\text{wi}_1, \text{st}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, \text{wi}_2, \text{st}_A) \leftarrow \mathcal{A}_1(1^\lambda, \text{wi}_1, z) \\ \text{aux} \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \\ \text{wi}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2) \\ \text{aux}' \leftarrow \text{WIPOK}.\hat{\text{P}}_{1.5}(1^\lambda, \beta) \end{array} \right] - \text{negl}(\lambda). \end{aligned} \quad (11)$$

Combining (10) with (11), we obtain (5). $\square$

5 Our 4-round SNMZK Argument

In this section, we prove our main result.

Theorem 4. Assume the existence of collision-resistant hash functions and injective one-way functions. Then, for any NP language L , there exists a 4-round adaptive-input interactive argument that satisfies one-many statistical non-malleable zero-knowledge with weak adaptive input selection.

To prove Theorem 4, we prove the following lemma.

Lemma 2. Assume the existence of the following building blocks.

- A 3-round synchronous non-malleable commitment scheme $NMC = (NMC.S_1, NMC.S_2, NMC.R)$ that satisfies the unique decommitment candidate property described in Theorem 2. Let $NMC.R_{D,weak}$ be the predicate that is guaranteed to exist by the unique decommitment candidate property.
- A 4-round public-coin adaptive-input argument $SWIAOK = (SWIAOK.P_1, SWIAOK.P_2, SWIAOK.V_1, SWIAOK.V_2, SWIAOK.V_D)$ that satisfies both adaptive-input computational special-soundness and adaptive-input statistical witness indistinguishability. Let $SWIAOK.Ext$ be the extractor that is guaranteed to exist by the adaptive-input computational special soundness.
- A 3-round adaptive-input OR-indistinguishable special-sound proof $WIPOK = (WIPOK.P_1, WIPOK.P_2, WIPOK.V, WIPOK.V_D)$ that satisfies the additional properties defined in Theorem 3. Let $WIPOK.Ext$ and $WIPOK.EQ$ be the extractor and predicate that are guaranteed to exist by the adaptive-input special soundness.

Then, for any NP language L , there exists a 4-round adaptive-input interactive argument that satisfies one-many statistical non-malleable zero-knowledge with weak adaptive input selection.

Since the building blocks listed in Lemma 2 can be constructed from any collision-resistant hash function and injective one-way function (cf. Theorem 2, Theorem 1, and Theorem 3), Lemma 2 implies Theorem 4. Thus, we focus on proving Lemma 2 below.

5.1 Protocol Description

The formal description of our 4-round SNMZK argument, $\langle P, V \rangle = (P_1, P_2, V_1, V_2, V_D)$, is given below.

4-round SNMZK argument $\langle P, V \rangle = (P_1, P_2, V_1, V_2, V_D)$

Let $(x, w) \in \mathbf{R}_L$ be an instance of an NP language L and a corresponding witness. Let $\lambda := |x|$ denote the security parameter. Let $\text{tag} \in \{0, 1\}^\lambda$ be a string representing the tag.

Round 1. $V_1(1^\lambda, \text{tag})$:

1. $\forall b \in \{0, 1\}$: Compute $(i\text{-nm}_{b,1}, i\text{-dec}_b, i\text{-st}_{nm,b}) \leftarrow NMC.S_1(1^\lambda, \text{tag}, r_b)$ for a random string $r_b \leftarrow \{0, 1\}^\lambda$.
2. $\forall b \in \{0, 1\}$: Compute $(o\text{-nm}_{b,1}, o\text{-dec}_b, o\text{-st}_{nm,b}) \leftarrow NMC.S_1(1^\lambda, \text{tag}, i\text{-dec}_b)$.
3. Compute $(wi_1, st_{wi}, aux_{wi}) \leftarrow WIPOK.\hat{P}_1(1^{\lambda_V}, \beta)$ for a random bit $\beta \leftarrow \{0, 1\}$, where λ_V is the length of the statements $x_{V,0}, x_{V,1} \in L_V$ defined in Round 3.
4. Compute $swi_1 \leftarrow SWIAOK.V_1(1^{\lambda_P})$, where λ_P is the length of the statement $x_P \in L_P$ defined in Round 4.
5. Output $\text{msg}_1 := (\{i\text{-nm}_{b,1}, o\text{-nm}_{b,1}\}_{b \in \{0,1\}}, wi_1, swi_1)$ and $st_V := (\{i\text{-st}_{nm,b}, o\text{-st}_{nm,b}\}_{b \in \{0,1\}}, st_{wi}, aux_{wi})$.

Round 2. $P_1(1^\lambda, \text{tag}, \text{msg}_1)$:

1. Compute the responses to $\{\text{i-nm}_{b,1}, \text{o-nm}_{b,1}\}_{b \in \{0,1\}}$, wi_1 , and swi_1 as follows.
 - $\forall b \in \{0, 1\}$: $\text{i-nm}_{b,2} \leftarrow \text{NMC.R}(1^\lambda, \text{tag}, \text{i-nm}_1)$.
 - $\forall b \in \{0, 1\}$: $\text{o-nm}_{b,2} \leftarrow \text{NMC.R}(1^\lambda, \text{tag}, \text{o-nm}_1)$.
 - $\text{wi}_2 \leftarrow \text{WIPOK.V}(1^{\lambda_V})$.
 - $(\text{swi}_2, \text{st}_{\text{swi}}) \leftarrow \text{SWIAOK.P}_1(1^{\lambda_P}, \text{swi}_1)$.
2. Output $\text{msg}_2 := (\{\text{i-nm}_{b,2}, \text{o-nm}_{b,2}\}_{b \in \{0,1\}}, \text{wi}_2, \text{swi}_2)$ and $\text{st}_P := \text{st}_{\text{swi}}$.

Round 3. $V_2(\text{st}_V, \text{msg}_2)$:

1. Compute the responses to $\{\text{i-nm}_{b,2}, \text{o-nm}_{b,2}\}_{b \in \{0,1\}}$ as follows.
 - $\forall b \in \{0, 1\}$: $\text{i-nm}_{b,3} \leftarrow \text{NMC.S}_2(\text{i-st}_{\text{nm},b}, \text{i-nm}_{b,2})$.
 - $\forall b \in \{0, 1\}$: $\text{o-nm}_{b,3} \leftarrow \text{NMC.S}_2(\text{o-st}_{\text{nm},b}, \text{o-nm}_{b,2})$.
2. Compute the response to wi_2 by $\text{wi}_3 \leftarrow \text{WIPOK.}\hat{\text{P}}_2(\text{st}_{\text{wi}}, \text{aux}_{\text{wi}}, x_{V,0}, x_{V,1}, w_{V,0}, w_{V,1}, \text{wi}_2)$, where $x_{V,b} := (\text{i-}\tau_b, \text{o-}\tau_b)$, $\text{i-}\tau_b := (\text{i-nm}_{b,1}, \text{i-nm}_{b,2}, \text{i-nm}_{b,3})$, $\text{o-}\tau_b := (\text{o-nm}_{b,1}, \text{o-nm}_{b,2}, \text{o-nm}_{b,3})$, and $w_{V,b} := (\text{i-dec}_b, \text{o-dec}_b)$ for each $b \in \{0, 1\}$. Here, $x_{V,b}$ is viewed as an instance of the following language $\mathbf{L}_V$.

$$\mathbf{L}_V := \left\{ (\text{i-}\tau, \text{o-}\tau) \mid \begin{array}{l} \exists (\text{i-dec}, \text{o-dec}) \text{ s.t.} \\ \text{NMC.R}_D(\text{i-}\tau, \text{i-dec}) \neq \perp \\ \wedge \text{NMC.R}_D(\text{o-}\tau, \text{o-dec}) = \text{i-dec} \neq \perp \end{array} \right\}.$$

3. Compute the response to swi_2 by $\text{swi}_3 \leftarrow \text{SWIAOK.V}_2(1^{\lambda_P})$.
4. Output $\text{msg}_3 := (\{\text{i-nm}_{b,3}, \text{o-nm}_{b,3}\}_{b \in \{0,1\}}, \text{wi}_3, \text{swi}_3)$.

Round 4. $P_2(\text{st}_P, x, w, \text{msg}_3)$:

1. Check whether $\text{WIPOK.V}_D(x_{V,0}, x_{V,1}, \tau_{\text{wi}}) \stackrel{?}{=} 1$, where $\tau_{\text{wi}} := (\text{wi}_1, \text{wi}_2, \text{wi}_3)$. If the check fails, abort.
2. Compute $\text{swi}_4 \leftarrow \text{SWIAOK.P}_2(\text{st}_{\text{swi}}, x_P, w, \text{swi}_3)$, where $x_P := (x, \text{i-nm}_{0,1}, \text{i-nm}_{1,1})$. Here, x_P is viewed as an instance of the language $\mathbf{L}_P := \{(x, \text{i-nm}_{0,1}, \text{i-nm}_{1,1}) \mid x \in \mathbf{L} \vee (\text{i-nm}_{0,1}, \text{i-nm}_{1,1}) \in \mathbf{L}_{\text{td}}\}$, where $\mathbf{L}_{\text{td}} := \{(\text{i-nm}_{0,1}, \text{i-nm}_{1,1}) \mid \exists (b, \text{dec}) \text{ s.t. } \text{NMC.R}_{D,\text{weak}}(\text{i-nm}_{b,1}, \text{dec}) = 1\}$.
3. Output $\text{msg}_4 := \text{swi}_4$.

Verification. $V_D(x, (\text{msg}_1, \dots, \text{msg}_4))$ outputs 1 if and only if $\text{SWIAOK.V}_D(x_P, \tau_{\text{swi}}) = 1$ for $\tau_{\text{swi}} := (\text{swi}_1, \text{swi}_2, \text{swi}_3, \text{swi}_4)$.

For each $b \in \{0, 1\}$, we call $\text{i-}\tau_b = (\text{i-nm}_{b,1}, \text{i-nm}_{b,2}, \text{i-nm}_{b,3})$ the b -th inner NMC commitment, and $\text{o-}\tau_b = (\text{o-nm}_{b,1}, \text{o-nm}_{b,2}, \text{o-nm}_{b,3})$ the b -th outer NMC commitment. The security of $\langle P, V \rangle$ is analyzed in the rest of this section.

5.2 Proofs of Completeness and Soundness

Since the delayed-input completeness can be verified by inspection, we focus on proving the adaptive-input soundness. Assume for contradiction that there exist a pair of PPT Turing machines $\mathcal{P} = (\mathcal{P}_1, \mathcal{P}_2)$ and a

polynomial p such that for infinitely many $\lambda \in \mathbb{N}$, there exist $\text{tag} \in \{0, 1\}^\lambda$ and $z \in \{0, 1\}^*$ such that

$$\Pr \left[\begin{array}{l} x \notin \mathbf{L} \\ \wedge V_D(x, \tau) = 1 \end{array} \middle| \begin{array}{l} (\text{msg}_1, \text{st}_V) \leftarrow V_1(1^\lambda, \text{tag}) \\ (\text{msg}_2, \text{st}_P) \leftarrow \mathcal{P}_1(1^\lambda, \text{tag}, \text{msg}_1, z) \\ \text{msg}_3 \leftarrow V_2(\text{st}_V, \text{msg}_2) \\ (x, \text{msg}_4) \leftarrow \mathcal{P}_2(\text{st}_P, \text{msg}_3) \\ \tau := (\text{msg}_1, \dots, \text{msg}_4) \end{array} \right] \geq \frac{1}{p(\lambda)}. \quad (12)$$

Fix any such $\mathcal{P}$, p , λ , tag , and z . Let $\varepsilon := 1/p(\lambda)$. Toward deriving a contradiction, we begin by considering the following experiment H_0 .

1. **Main thread.** Run V_1 , $\mathcal{P}_1$, V_2 , and $\mathcal{P}_2$ as described in (12) to obtain a statement x and a transcript $\tau = (\text{msg}_1, \text{msg}_2, \text{msg}_3, \text{msg}_4)$. If $V_D(x, \tau) \neq 1$, abort. Let $(\text{msg}_2, \text{st}_P)$ denote the output of $\mathcal{P}_1$.
2. **Look-ahead thread.** Define x' , msg'_3 , and msg'_4 as follows: parse msg_3 as $\text{msg}_3 = (\{i\text{-nm}_{b,3}, o\text{-nm}_{b,3}\}_{b \in \{0,1\}}, \text{wi}_3, \text{swi}_3)$; then, compute $\text{swi}'_3 \leftarrow \text{SWIAOK.V}_2(1^{\lambda_P})$, let $\text{msg}'_3 := (\{i\text{-nm}_{b,3}, o\text{-nm}_{b,3}\}_{b \in \{0,1\}}, \text{wi}_3, \text{swi}'_3)$, and compute $(x', \text{msg}'_4) \leftarrow \mathcal{P}_2(\text{st}_P, \text{msg}'_3)$. If $V_D(x', \tau') \neq 1$ for $\tau' := (\text{msg}_1, \text{msg}_2, \text{msg}'_3, \text{msg}'_4)$, abort.
3. Output $w := \text{SWIAOK.Ext}(x, \tau, x', \tau')$.

Claim 1. Let β denote the bit used by V_1 for $\text{WIPOK}.\hat{P}_1$ in Round 1, and let $\{i\text{-nm}_{b,1}\}_{b \in \{0,1\}}$ denote the NMC messages sent by V_1 in Round 1. Let us say that event *BAD* occurs in H_0 if the output w is a valid witness for $(i\text{-nm}_{0,1}, i\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}$ and of the form $(1 - \beta, *)$. Then, when λ is sufficiently large, event *BAD* occurs with probability at least $\varepsilon_0 := \varepsilon^3/32$ in H_0 .

Proof. It follows from (12) and an averaging argument that we have $x \notin \mathbf{L} \wedge V_D(x, \tau) = 1 \wedge x' \notin \mathbf{L} \wedge V_D(x', \tau') = 1$ with probability at least $(\varepsilon/2)^3$ in H_0 . Thus, from the adaptive-input computational special soundness of SWIAOK , there exists a negligible function negl such that

$$\Pr[w \text{ is a valid witness for } (i\text{-nm}_{0,1}, i\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}] \geq (\varepsilon/2)^3 - \text{negl}(\lambda).$$

Thus, from the adaptive-input OR-indistinguishability of the hybrid prover of WIPOK , there exists a negligible function negl' such that

$$\Pr \left[\begin{array}{l} w \text{ is a valid witness for } (i\text{-nm}_{0,1}, i\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}} \\ \text{and of the form } (1 - \beta, *) \end{array} \right] \geq \frac{(\varepsilon/2)^3 - \text{negl}'(\lambda)}{2}.$$

Thus, when λ is sufficiently large, event *BAD* occurs with probability at least $\varepsilon^3/32$ in H_0 . $\square$

Next, we consider the following hybrid experiments.

- Hybrid H_1 is identical to H_0 except that in V_1 and V_2 , the WIPOK messages wi_1 and wi_3 are computed by $(\text{wi}_1, \text{st}_{\text{wi}}) \leftarrow \text{WIPOK.P}_1(1^{\lambda_V}, 1^{\lambda_V}, \beta)$ and $\text{wi}_3 \leftarrow \text{WIPOK.P}_2(\text{st}_{\text{wi}}, x_{V,0}, x_{V,1}, w_{V,\beta}, \text{wi}_2)$ using the real prover algorithms (rather than by $(\text{wi}_1, \text{st}_{\text{wi}}, \text{aux}_{\text{wi}}) \leftarrow \text{WIPOK}.\hat{P}_1(1^{\lambda_V}, \beta)$ and $\text{wi}_3 \leftarrow \text{WIPOK}.\hat{P}_2(\text{st}_{\text{wi}}, \text{aux}_{\text{wi}}, x_{V,0}, x_{V,1}, w_{V,0}, w_{V,1}, \text{wi}_2)$ using the hybrid prover algorithms).
- Hybrid H_2 is identical to H_1 except that in V_1 , the $(1 - \beta)$ -th outer NMC commitment is computed by committing to a random string (i.e., by running $\text{NMC.S}_1(1^\lambda, \text{tag}, r')$ for random $r' \in \{0, 1\}^\lambda$ rather than $\text{NMC.S}_1(1^\lambda, \text{tag}, i\text{-dec}_{1-\beta})$).

Claim 2. In H_1 , event *BAD* occurs with probability at least $\varepsilon_1 := \varepsilon_0 - \text{negl}(\lambda)$.

Proof. This claim follows directly from the indistinguishability of the real and hybrid provers of WIPOK . $\square$

Claim 3. In H_2 , event *BAD* occurs with probability at least $\varepsilon_2 := \varepsilon_1 - \text{negl}(\lambda)$.

Proof. This claim follows directly from the hiding property of NMC. □

Now, we derive a contradiction by breaking the hiding property of NMC using H_2 . Toward this end, it suffices to consider a cheating receiver that internally emulates H_2 while using the external committer to generate the $(1 - \beta)$ -th inner NMC commitment. (The committed value is either the random string $r_{1-\beta}$ that the cheating receiver selected or an independent random string.) Since this cheating receiver perfectly emulates the experiment H_2 , event *BAD* occurs in the internally emulated H_2 with non-negligible probability. Furthermore, when event *BAD* occurs in the internally emulated H_2 , the cheating receiver can obtain the value that the external committer committed to.¹⁹ Thus, the cheating receiver can break the hiding property of NMC.

5.3 Description of Simulator-Extractor

We proceed to the proof of one-many statistical non-malleable zero-knowledge with weak adaptive input selection.

As explained in Section 2, our simulator-extractor simulates the left interaction using the extractor of WIPOK and extracts a witness in each accepting right interaction using the extractor of SWIAOK. Before describing how it works formally, let us mention two technical details omitted in Section 2. First, when the extractor of WIPOK is used, our simulator-extractor keeps rewinding the adversary until it obtains two (rather than one) additional accepting transcripts, and the extractor is invoked on these two additional transcripts. At a high level, using the extractor in this way makes the rewinding process almost independent of the rest of the simulator-extractor, thereby making it easy to use Lemma 1 (the technical lemma shown on WIPOK in Section 4).²⁰ Second, when the extractor of SWIAOK is used for each right interaction, our simulator-extractor keeps rewinding the adversary until it obtains an additional accepting transcript in which the left interaction is completed in the same way as before. (For example, if the left interaction was completed using the 0-th inner NMC commitment for SWIAOK before the rewinding, our simulator-extractor tries to obtain an accepting transcript in which the left interaction is again completed using the 0-th inner NMC commitment for SWIAOK.) Using the extractor in this way simplifies the analysis of witness extractability.

Formally, for any PPT man-in-the-middle adversary $\mathcal{A}$ and a polynomial m such that $\mathcal{A}$ participates in at most $m(\lambda)$ right interactions, our simulator-extractor, $\mathcal{SE}$, works as follows.

Simulator-extractor $\mathcal{SE}$.

Simulation part. On input $(1^\lambda, z)$, the simulator-extractor $\mathcal{SE}$ internally invokes $\mathcal{A}(1^\lambda, z)$ and emulates the left and right interactions for $\mathcal{A}$ in the following way.

In each right interaction, $\mathcal{SE}$ interacts with $\mathcal{A}$ using the honest verifier algorithms. That is, $\mathcal{SE}$ interacts with $\mathcal{A}$ in the i -th right interaction as follows. (In what follows, we denote Round j of the i -th right interaction by Round $i-j$ for each $j \in \{1, \dots, 4\}$.)

Round $i-1$. Receiving $\widetilde{\text{tag}}_i$, compute $(\widetilde{\text{msg}}_{i,1}, \widetilde{\text{st}}_{V,i}) \leftarrow V_1(1^\lambda, \widetilde{\text{tag}}_i)$ and send $\widetilde{\text{msg}}_{i,1}$ to $\mathcal{A}$.

Round $i-2$. Receiving $\widetilde{\text{msg}}_{i,2}$, compute $\widetilde{\text{msg}}_{i,3} \leftarrow V_2(\widetilde{\text{st}}_{V,i}, \widetilde{\text{msg}}_{i,2})$ and send $\widetilde{\text{msg}}_{i,3}$ to $\mathcal{A}$.

In the left interaction, $\mathcal{SE}$ interacts with $\mathcal{A}$ as follows.

Round 2. Receiving $(\text{tag}, \text{msg}_1)$, compute $(\text{msg}_2, \text{st}_P) \leftarrow P_1(1^\lambda, \text{tag}, \text{msg}_1)$ and send msg_2 to $\mathcal{A}$.

Round 4. Receiving (x, msg_3) , do the following.

¹⁹Formally, the soundness of the unique decommitment candidate property of NMC guarantees that when *BAD* occurs, any dec such that $\text{NMC.RD}_{\text{weak}}(\text{nm}_{1-\beta,1}, \text{dec}) = 1$ is a valid decommitment of the $(1 - \beta)$ -th inner NMC commitment. Thus, it can be used to obtain the value the external committer committed to.

²⁰Specifically, the rewinding process only determines which of the two inner NMC commitments are used by the simulator-extractor for the simulation of the SWIAOK proof.

4:1 Like the honest prover, check whether msg_3 is accepting, i.e., whether $\text{WIPOK.V}_D(x_{V,0}, x_{V,1}, \tau_{\text{wi}}) \stackrel{?}{=} 1$ for the WIPOK statements $x_{V,0}, x_{V,1}$ and transcript τ_{wi} defined based on $(\text{msg}_1, \text{msg}_2, \text{msg}_3)$. If the check fails, abort the left interaction.

4:2 If the check passes, repeat the following until the condition described below is met.

Left rewinding. Rewind $\mathcal{A}$ to the point just before sending msg_2 to $\mathcal{A}$ in Round 2. Then, sample a new Round-2 message msg'_2 using $P_1(1^\lambda, \text{tag}, \text{msg}_1)$ with fresh randomness, send msg'_2 to $\mathcal{A}$, and continue interacting with $\mathcal{A}$ until obtaining a new Round-3 message msg'_3 . (During the interaction with $\mathcal{A}$, the right interactions are simulated as before using fresh randomness.)

The condition for breaking the loop is to find *two* new accepting Round-3 messages $\text{msg}'_3, \text{msg}''_3$. Let msg'_2 denote the Round-2 message that is sent to $\mathcal{A}$ when obtaining msg'_3 . Let $(x'_{V,0}, x'_{V,1})$ and τ'_{wi} denote the WIPOK statements and transcript defined based on $(\text{msg}_1, \text{msg}'_2, \text{msg}'_3)$. Let $\text{msg}''_2, (x''_{V,0}, x''_{V,1})$, and τ''_{wi} be defined similarly w.r.t. msg''_3 . (At the end of this step, the state of $\mathcal{A}$ is restored to the one that $\mathcal{A}$ had before the above rewinding.)

4:3 Compute $w_V := \text{WIPOK.Ext}((x'_{V,0}, x'_{V,1}), \tau'_{\text{wi}}, (x''_{V,0}, x''_{V,1}), \tau''_{\text{wi}})$. If $\exists \beta \in \{0, 1\}$ s.t. $(x'_{V,\beta}, w_V) \in \mathbf{R}_{L_V}$, parse w_V as (i-dec, o-dec) and let $w_{\text{td}} := (\beta, \text{i-dec})$. If there is no such β , output fail and abort.

4:4 Compute $\text{swi}_4 \leftarrow \text{SWIAOK.P}_2(\text{st}_{\text{swi}}, x_P, w_{\text{td}}, \text{swi}_3)$ by using w_{td} with the SWIAOK prover state st_{swi} , statement x_P , and challenge swi_3 that are defined based on $(x, \text{msg}_1, \text{msg}_2, \text{msg}_3)$. Then, send $\text{msg}_4 := \text{swi}_4$ to $\mathcal{A}$.

Let *sim-view* be the joint view of $\mathcal{A}$ and the honest verifiers emulated in the above. We call the execution of $\mathcal{A}$ recorded in *sim-view* the *main thread*, and the executions of $\mathcal{A}$ generated during the left rewinding the *left look-ahead threads*. We call w_{td} (the witness used for SWIAOK in Step 4:4) the *trapdoor witness*.

Extraction part. After completing the simulation part, $\mathcal{SE}$ defines candidate witnesses $\{\tilde{w}_i\}_{i \in [m(\lambda)]}$ in the following way. For each $i \in [m(\lambda)]$ such that the i -th right interaction is rejecting or uses the same tag as the left interaction, $\mathcal{SE}$ lets $\tilde{w}_i := \perp$. For every other i , $\mathcal{SE}$ does the following. Let $\tilde{x}_i$ and $\widetilde{\text{msg}}_{i,4}$ denote the statement and message sent by $\mathcal{A}$ in Round $i-4$.

i :1 Repeat the following until the condition described below is met.

Right rewinding. Rewind $\mathcal{A}$ to the point just before sending $\widetilde{\text{msg}}_{i,3}$ to $\mathcal{A}$ in Round $i-3$. Then, define a new Round-3 message $\widetilde{\text{msg}}'_{i,3}$ as follows: parse $\widetilde{\text{msg}}_{i,3}$ as $\widetilde{\text{msg}}_{i,3} = (\{\widetilde{\text{i-nm}}_{b,3}, \widetilde{\text{o-nm}}_{b,3}\}_{b \in \{0,1\}}, \widetilde{\text{wi}}_3, \widetilde{\text{swi}}_3)$; then, sample a new challenge message $\widetilde{\text{swi}}'_3 \leftarrow \text{SWIAOK.V}_2(1^{\lambda_P})$ for SWIAOK and let $\widetilde{\text{msg}}'_{i,3} := (\{\widetilde{\text{i-nm}}_{b,3}, \widetilde{\text{o-nm}}_{b,3}\}_{b \in \{0,1\}}, \widetilde{\text{wi}}_3, \widetilde{\text{swi}}'_3)$. Then, send $\widetilde{\text{msg}}'_{i,3}$ to $\mathcal{A}$ and continue interacting with $\mathcal{A}$ until obtaining a new Round 4 message $\widetilde{\text{msg}}'_{i,4}$ along with a new statement $\tilde{x}'_i$ in Round $i-4$. (During the interaction with $\mathcal{A}$, the left and other right interactions are simulated as before using fresh randomness, including the left rewinding in Step 4:2.)

Let us call the interactions with $\mathcal{A}$ during the right rewinding the *right look-ahead threads*. Then, roughly speaking, the condition for breaking the loop is to find a right look-ahead thread in which the i -th right interaction is accepting *and* the left interaction is completed in the same way as in the main thread. Formally, the condition for breaking the loop is defined as follows. In the main and right look-ahead threads, for each $b \in \{0, 1\}$, let EVENT_b denote the event that (i) the i -th right interaction is accepting and (ii) Round 4 of the left interaction is scheduled after Round $i-3$ and is completed using a witness of the form $(b, *)$ for SWIAOK. Similarly, let EVENT_2 denote the

event that (i) the i -th right interaction is accepting and (ii) either Round 4 of the left interaction is scheduled before Round $i-3$ or the left interaction is aborted. Let $k^* \in \{0, 1, 2\}$ be such that event EVENT_{k^*} occurs in the main thread. Then, the condition for breaking the loop is to find a right look-ahead thread in which event EVENT_{k^*} occurs.

$i:2$ Compute $\tilde{w}_{P,i} := \text{SWIAOK.Ext}(\tilde{x}_{P,i}, \tilde{\tau}_{\text{swi},i}, \tilde{x}'_{P,i}, \tilde{\tau}'_{\text{swi},i})$ for (i) the SWIAOK statement $\tilde{x}_{P,i}$ and transcript $\tilde{\tau}_{\text{swi},i}$ defined based on $(\tilde{x}_i, \widetilde{\text{msg}}_{i,1}, \widetilde{\text{msg}}_{i,2}, \widetilde{\text{msg}}_{i,3}, \widetilde{\text{msg}}_{i,4})$ and (ii) the SWIAOK statement $\tilde{x}'_{P,i}$ and transcript $\tilde{\tau}'_{\text{swi},i}$ defined based on $(\tilde{x}'_i, \widetilde{\text{msg}}_{i,1}, \widetilde{\text{msg}}_{i,2}, \widetilde{\text{msg}}'_{i,3}, \widetilde{\text{msg}}'_{i,4})$. If $(\tilde{x}_{P,i}, \tilde{w}_{P,i}) \in \mathbf{R}_{LP}$, let $\tilde{w}_i := \tilde{w}_{P,i}$. Otherwise, let $\tilde{w}_i := \perp_{\text{fail}}$.

Final output. After completing the extraction part, $\mathcal{SE}$ outputs $(\text{sim-view}, \{\tilde{w}_i\}_{i \in [m(\lambda)]})$.

We analyze $\mathcal{SE}$ in the rest of this section. Fix any PPT man-in-the-middle adversary $\mathcal{A}$ and a polynomial m such that $\mathcal{A}$ participates in at most $m(\lambda)$ right interactions. Fix any PPT witness-selecting machine M .

5.4 Running Time of Simulator-Extractor

First, we obtain an upper bound for the expected running time of the simulation part of $\mathcal{SE}$. Toward this end, it suffices to analyze the number of the left look-ahead threads generated during the simulation part. Let R be the random variable representing the randomness used by $\mathcal{SE}$ until the point just before sampling a Round-2 message msg_2 for $\mathcal{A}$ in the left interaction in the main thread, and fix arbitrary randomness r . Let p denote the probability that $\mathcal{A}$ outputs an accepting Round-3 message msg_3 in the left interaction in the main thread under the condition $R = r$. Then, since each left look-ahead thread is generated identically to the main thread, it follows that under the condition $R = r$, the expected number of the left look-ahead threads generated during the simulation part is bounded from the above by $p \cdot 2/p = 2$. Since r is arbitrary, the expected number of left look-ahead threads during the simulation part is bounded from the above by 2.

Next, we obtain an upper bound for the expected running time of the extraction part of $\mathcal{SE}$. Toward this end, it suffices to analyze the time spent in the right rewinding in each accepting right interaction. Fix any $i \in [m(\lambda)]$. Let R be the random variable representing the randomness used by $\mathcal{SE}$ until the point just before sampling a SWIAOK challenge message $\widetilde{\text{swi}}_{i,3}$ in Round $i-3$ in the main thread, and fix arbitrary randomness r . Let $T_{\text{sim}}(r)$ denote the expected running time of the simulation part of $\mathcal{SE}$ under the condition $R = r$. For each $k \in \{0, 1, 2\}$, let p_k denote the probability that event EVENT_k occurs in the simulation part of $\mathcal{SE}$ under the condition $R = r$. For each $j > 0$, let X_j denote the random variable representing the time spent in the j -th right look-ahead thread of the i -th right interaction when $R = r$. Then, under the condition $R = r$, the expected time spent in the right rewinding of the i -th right interaction is

$$\begin{aligned} T_{\text{ext}}(r) &:= \mathbb{E} \left[\sum_{j=1}^{\infty} X_j \right] = \sum_{j=1}^{\infty} \mathbb{E} [X_j] = \sum_{j=1}^{\infty} \sum_{k=1}^3 p_k (1 - p_k)^{j-1} T_{\text{sim}}(r) = \sum_{k=1}^3 p_k \cdot \frac{1}{p_k} \cdot T_{\text{sim}}(r) \\ &= 3 \cdot T_{\text{sim}}(r). \end{aligned}$$

Recalling that the expected running time of the simulation part of $\mathcal{SE}$ is bounded by $\text{poly}(\lambda)$ (and therefore $\mathbb{E}_r [T_{\text{sim}}(r)] \leq \text{poly}(\lambda)$), the expected time spent in the right rewinding of the i -th right interaction is

$$\mathbb{E}_r [T_{\text{ext}}(r)] = 3 \mathbb{E}_r [T_{\text{sim}}(r)] \leq \text{poly}(\lambda).$$

5.5 Statistical Simulation Property of Simulator-Extractor

For any $\lambda \in \mathbb{N}$ and $(z, z') \in Z_{\mathcal{A}, M}(\lambda)$, consider the following hybrid experiments H_0, H_1, H_2 .

- Hybrid H_0 is identical to the real man-in-the-middle experiment. That is, $\mathcal{A}(1^\lambda, z)$ interacts with a single honest prover and $m(\lambda)$ honest verifiers as described in Section 3.4 with the help of the witness-selecting machine $M(\cdot, \cdot, z')$. The output of the experiment is the joint view of $\mathcal{A}$ and the honest verifiers.
- Hybrid H_1 is an experiment where the simulator-extractor $\mathcal{SE}(1^\lambda, z)$ is executed with the following modifications.

1. Only the simulation part is executed.
2. In Step 4:4 of the simulation part, the SWIAOK response message swi_4 is computed as in the honest prover algorithm, i.e., by $\text{swi}_4 \leftarrow \text{SWIAOK.P}_2(\text{st}_{\text{swi}}, x_P, w, \text{swi}_3)$ using the witness w provided by M .²¹

The output of the experiment is the simulated view sim-view computed in $\mathcal{SE}$.

- Hybrid H_2 is identical to H_1 except that Step 4:4 of the simulation part is executed as in $\mathcal{SE}$, i.e., swi_4 is computed by $\text{swi}_4 \leftarrow \text{SWIAOK.P}_2(\text{st}_{\text{swi}}, x_P, w_{\text{td}}, \text{swi}_3)$ using the witness w_{td} that is extracted through the left rewinding.

First, we observe that the outputs of H_0 and H_1 are statistically indistinguishable. Toward this end, it suffices to show that in H_1 , the simulator-extractor $\mathcal{SE}$ outputs fail in Step 4:3 of the simulation part with negligible probability. Recall that in Step 4:3 of the simulation part, $\mathcal{SE}$ outputs fail only if $\text{WIPOK.Ext}((x'_{V,0}, x'_{V,1}), \tau'_{\text{wi}}, (x''_{V,0}, x''_{V,1}), \tau''_{\text{wi}})$ fails to output a valid witness for $x'_{V,0} \in \mathbf{L}_V \vee x'_{V,1} \in \mathbf{L}_V$, which in turn occurs only if the two challenge messages $\text{wi}'_2, \text{wi}''_2$ in the WIPOK transcripts $\tau'_{\text{wi}}, \tau''_{\text{wi}}$ satisfy $\text{WIPOK.EQ}(\text{wi}'_{b,2}, \text{wi}''_{b,2}) = 1$. Given that the number of the left look-ahead threads, denoted by N_{left} , exceeds $2 \cdot 2^{\lambda_V/2}$ with probability at most $2^{-\lambda_V/2}$,²² the delayed-input special soundness of WIPOK implies that we have $\text{WIPOK.EQ}(\text{wi}'_{b,2}, \text{wi}''_{b,2}) = 1$ with probability at most

$$\begin{aligned} & \Pr \left[N_{\text{left}} > 2 \cdot 2^{\lambda_V/2} \right] + \Pr \left[\text{WIPOK.EQ}(\text{wi}'_{b,2}, \text{wi}''_{b,2}) = 1 \mid N_{\text{left}} \leq 2 \cdot 2^{\lambda_V/2} \right] \\ & \leq 2^{-\lambda_V/2} + 2 \cdot 2^{\lambda_V/2} \cdot 2^{-\lambda_V} \leq \text{negl}(\lambda), \end{aligned}$$

where the last inequality follows from $\lambda_V = \text{poly}(\lambda)$.

Next, we observe that the outputs of H_1 and H_2 are statistically indistinguishable. First, let us observe that in H_2 , the witness used for SWIAOK in Step 4:4 of the simulation part is indeed a valid witness. When $\mathcal{SE}$ does not abort in Step 4:3 of the simulation part, the witness $w_{\text{td}} = (\beta, \text{i-dec})$ satisfies $\text{NMC.RD}(\text{i-}\tau'_\beta, \text{i-dec}) \neq \perp$ for the β -th inner NMC commitment $\text{i-}\tau'_\beta = (\text{i-nm}_{\beta,1}, \text{i-nm}'_{\beta,2}, \text{i-nm}'_{\beta,3})$ that is contained in $(\text{msg}_1, \text{msg}'_2, \text{msg}'_3)$. Thus, by the correctness of the unique decommitment candidate property of NMC, we have $\text{NMC.RD}_{\text{weak}}(\text{i-nm}_{\beta,1}, \text{i-dec}) = 1$, and therefore, w_{td} is a valid witness for SWIAOK in the main thread. Then, since H_2 differs from H_1 only in the witness used for SWIAOK, the indistinguishability follows directly from the adaptive-input statistical witness indistinguishability of SWIAOK.²³

Now, since the output of H_0 is identical to the view in the real man-in-the-middle experiment whereas the output of H_2 is identical to the view simulated by $\mathcal{SE}$, the statistical simulation property of $\mathcal{SE}$ follows.

5.6 Witness Extractability of Simulator-Extractor

Finally, we analyze the witness extractability of $\mathcal{SE}$. For each $i \in [m(\lambda)]$, let us say that event BAD_i occurs in the execution of $\mathcal{SE}$ if (i) $\mathcal{SE}$ does not abort, (ii) in the main thread, the i -th right interaction is accepting and

²¹That is, the witness w is obtained by internally running M on input the statement chosen by $\mathcal{A}$, the current view of $\mathcal{A}$, and auxiliary input z' .

²²This follows from Markov's inequality since the expected number of the left look-ahead threads is at most 2.

²³To see that the left rewinding does not interfere with the adaptive-input statistical witness indistinguishability of SWIAOK, observe that the left rewinding only involves sampling new commit messages for SWIAOK (which can be done by anyone knowing the statement length).

uses a different tag from the left interaction, and (iii) the i -th extracted witness $\tilde{w}_i$ is not a valid witness for the i -th right statement $\tilde{x}_i$. Assume for contradiction that there exists a polynomial q such that for infinitely many $\lambda \in \mathbb{N}$, there exist $(z, z') \in Z_{\mathcal{A}, M}(\lambda)$ and $i^* \in [m(\lambda)]$ such that BAD_{i^*} occurs with probability at least $1/q(\lambda)$. Fix any such $q, \lambda, (z, z')$, and i^* . For simplicity (and without loss of generality), we assume that $\mathcal{A}$ schedules the messages in the left and right interactions in the following way.

- Round 4 of every right interaction is scheduled at the end of the experiment.
- Round 2 of the left interaction is scheduled immediately after Round 1 of the left interaction, and Round 4 of the left interaction is scheduled immediately after Round 3 of the left interaction.²⁴
- Round 1 of each right interaction is scheduled immediately after the point where $\mathcal{A}$ selects a tag for the corresponding right interaction, and Round 3 of each right interaction is scheduled immediately after Round 2 of the corresponding right interaction.

Under this assumption, the i^* -th right interaction must be scheduled in one of the following ways (see Figure 4).²⁵

Schedule 1. Rounds 1 and 2 of the left interaction are scheduled between Rounds i^*-1 and i^*-2 , and Rounds 3 and 4 of the left interaction are scheduled between Rounds i^*-3 and i^*-4 . (In other words, the i^* -th right interaction is synchronized with the left interaction.)

Schedule 2. Round 2 of the left interaction is scheduled before the beginning of the i^* -th right interaction, and Rounds 3 and 4 of the left interaction are scheduled between Rounds i^*-3 and i^*-4 .

Schedule 3. Round 4 of the left interaction is scheduled before Round i^*-2 .

Schedule 4. Round 1 of the left interaction is scheduled after Round i^*-3 .

For each $j \in \{1, \dots, 4\}$, let $\text{BAD}_{i^*, j}$ denote the event that (i) BAD_{i^*} occurs and (ii) the i^* -th right interaction is scheduled in Schedule j in the main thread. Since we have $\Pr[\text{BAD}_{i^*}] \geq 1/q(\lambda)$, there exists $j^* \in \{1, \dots, 4\}$ such that $\Pr[\text{BAD}_{i^*, j^*}] \geq 1/4q(\lambda)$.

5.6.1 Synchronous Case ($j^* = 1$)

First, we consider the case that $\Pr[\text{BAD}_{i^*, 1}] \geq 1/4q(\lambda)$. We start by making several simplifications to $\mathcal{SE}$. Consider the following hybrid simulator-extractors $\mathcal{SE}_1, \dots, \mathcal{SE}_5$.

1. $\mathcal{SE}_1$ is identical to $\mathcal{SE}$ except that $\mathcal{SE}_1$ outputs fail and aborts if the i^* -th right interaction is not scheduled in Schedule 1 in the main thread.
2. $\mathcal{SE}_2$ is identical to $\mathcal{SE}_1$ except that in the extraction part, $\mathcal{SE}_2$ lets $\tilde{w}_i := \perp$ for every $i \neq i^*$ and skips every step related to such $\tilde{w}_i$ (i.e., Steps [i:1](#) and [i:2](#) for every $i \neq i^*$).
3. $\mathcal{SE}_3$ is identical to $\mathcal{SE}_2$ except that when defining the i^* -th witness $\tilde{w}_{i^*}$ in the extraction part, $\mathcal{SE}_3$ only generates a single right look-ahead thread (rather than keep generating right look-ahead threads until EVENT_{k^*} occurs in the current right look-ahead thread, where recall that $k^* \in \{0, 1, 2\}$ is such that EVENT_{k^*} occurs in the main thread). If EVENT_{k^*} occurs in the unique right look-ahead thread, $\mathcal{SE}_3$ defines $\tilde{w}_{i^*}$ as in $\mathcal{SE}_2$. Otherwise, $\mathcal{SE}_3$ outputs fail and aborts.

²⁴In other words, no messages belonging to the right interactions are scheduled between Rounds 1 and 2 (resp., 3 and 4) of the left interaction.

²⁵When the left interaction is aborted, we think the left interaction is completed at the point of being aborted by exchanging empty messages for the remaining rounds.

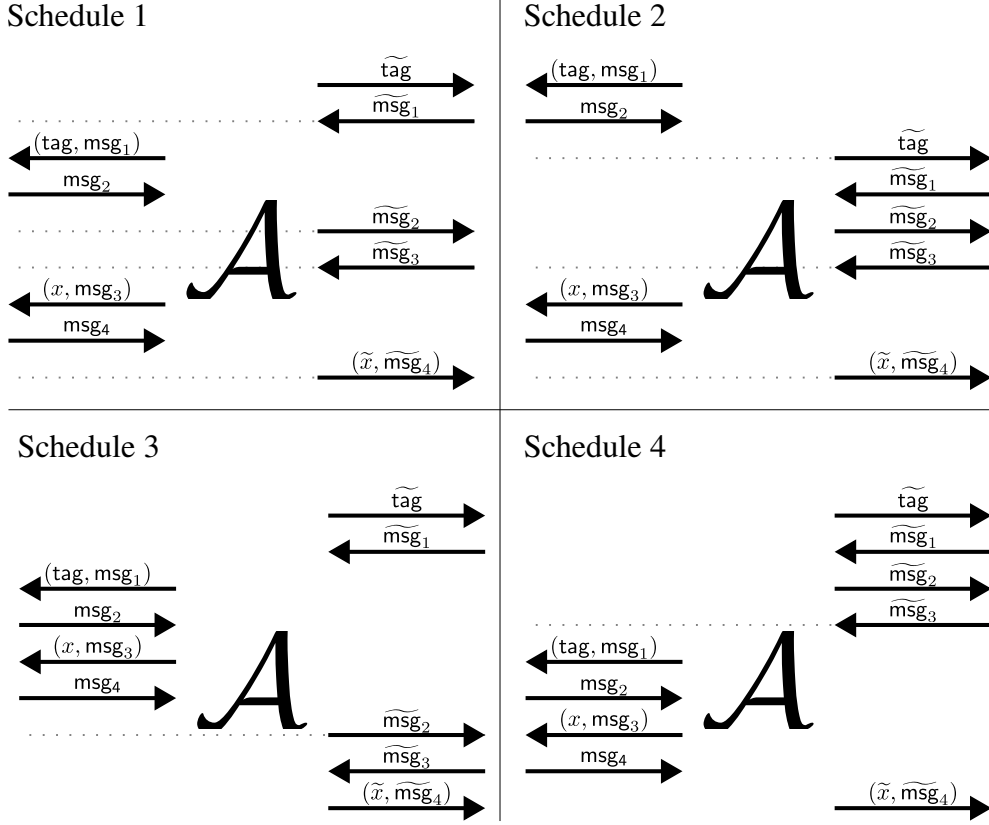


Figure 4: Four possible message schedules among the messages in the left and the i^* -th right interactions.

4. $\mathcal{SE}_4$ is identical to $\mathcal{SE}_3$ except that in the right look-ahead thread, $\mathcal{SE}_4$ completes SWIAOK in the left interaction by reusing the witness that it used in the main thread. Specifically, consider the case that EVENT_0 or EVENT_1 occurs in the main thread. From the definitions of EVENT_0 and EVENT_1 , it follows that in the main thread, Round 4 of the left interaction is successfully completed after Round i^*-3 by extracting a witness w_{td} for SWIAOK using the left rewinding and WIPOK.Ext. Then, when completing Round 4 of the left interaction in the right look-ahead thread, $\mathcal{SE}_4$ no longer re-executes the left rewinding and WIPOK.Ext to obtain a (potentially different) witness for SWIAOK; instead, it simply reuses w_{td} . Furthermore, in the right look-ahead thread, $\mathcal{SE}_4$ no longer checks whether EVENT_{k^*} occurs. Instead, $\mathcal{SE}_4$ checks whether a new accepting message is obtained in Round i^*-4 : if yes, $\mathcal{SE}_4$ defines the i^* -th witness $\widetilde{w}_{i^*}$ as in $\mathcal{SE}_3$ using SWIAOK.Ext; if no, $\mathcal{SE}_4$ outputs fail and aborts. ($\mathcal{SE}_4$ also outputs fail and aborts if w_{td} is not a valid witness for SWIAOK in the right look-ahead thread.)
5. $\mathcal{SE}_5$ is identical to $\mathcal{SE}_4$ except that $\mathcal{SE}_5$ only generates two left look-ahead threads in the simulation part (rather than keep generating left look-ahead threads until obtaining two new accepting Round-3 messages). If two new accepting Round-3 messages are obtained in the two left look-ahead threads, $\mathcal{SE}_5$ completes the simulation part as in $\mathcal{SE}_4$. Otherwise, $\mathcal{SE}_5$ outputs fail and aborts.

The full description of the final hybrid simulator-extractor $\mathcal{SE}' := \mathcal{SE}_5$ is provided in [Appendix B](#). It is not difficult to see that $\text{BAD}_{i^*,1}$ still occurs with non-negligible probability in $\mathcal{SE}'$. For completeness, we prove the following in [Appendix C](#).

Claim 4. *There exists a polynomial q' (determined based on q) such that event $\text{BAD}_{i^*,1}$ occurs with probability at least $1/q'(\lambda)$ in $\mathcal{SE}'$.*

Since the extraction part is only performed for the i^* -th right interaction in $\mathcal{SE}'$, in what follows we refrain from mentioning i^* in our notation. (For example, when we say “the right interaction,” we refer to the i^* -th

right interaction, and we write BAD_1 rather than $\text{BAD}_{i^*,1}$.) We comment that $\mathcal{SE}'$ runs in strict polynomial time because it generates only a fixed number of threads.

Next, we observe that the extracted witness $\tilde{w}$ is computationally independent of the bit $\tilde{\beta}$ that $\mathcal{SE}'$ uses for $\text{WIPOK}.\hat{P}_1$ in Round 1 of the right interaction.

Claim 5. *Let $\tilde{\beta}$ denote the bit used by $\mathcal{SE}'$ for $\text{WIPOK}.\hat{P}_1$ in Round 1 of the right interaction, and let $\{\tilde{i}\text{-nm}_{b,1}\}_{b \in \{0,1\}}$ denote the NMC messages sent by $\mathcal{SE}'$ in Round 1 of the right interaction. Let us say that event $\widehat{\text{BAD}}$ occurs in an execution of $\mathcal{SE}'$ if (i) BAD_1 occurs and (ii) the extracted witness $\tilde{w}$ is a valid witness for $(\tilde{i}\text{-nm}_{0,1}, \tilde{i}\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}$ and of the form $(1 - \tilde{\beta}, *)$. Then, there exists a polynomial $\hat{q}$ (determined based on q') such that when λ is sufficiently large, event $\widehat{\text{BAD}}$ occurs with probability at least $1/\hat{q}(\lambda)$.*

Proof. We show that $\widehat{\text{BAD}}$ occurs with probability roughly half of the probability that BAD_1 occurs. By definition, when BAD_1 occurs (and therefore the extracted witness $\tilde{w}$ is not a valid witness for $\tilde{x} \in \mathbf{L}$), either $\tilde{w} = \perp_{\text{fail}}$ or $\tilde{w}$ is a valid witness for $(\tilde{i}\text{-nm}_{0,1}, \tilde{i}\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}$. First, we have $p_{\text{fail}} := \Pr[\text{BAD}_1 \wedge \tilde{w} = \perp_{\text{fail}}] \leq \text{negl}(\lambda)$ because of the adaptive-input computational special soundness of SWIAOK.²⁶ To analyze the case that $\tilde{w}$ is a valid witness for $(\tilde{i}\text{-nm}_{0,1}, \tilde{i}\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}$, let us define p (resp., p') to be the probability that (i) BAD_1 occurs and (ii) $\tilde{w}$ is a valid witness for $(\tilde{i}\text{-nm}_{0,1}, \tilde{i}\text{-nm}_{1,1}) \in \mathbf{L}_{\text{td}}$ and of the form $(1 - \tilde{\beta}, *)$ (resp., $(\tilde{\beta}, *)$). Then, we have $|p - p'| \leq \text{negl}(\lambda)$ because of the adaptive-input rewinding OR-indistinguishability of the hybrid prover of WIPOK.²⁷ Now, using $p_{\text{fail}} + p + p' = \Pr[\text{BAD}_1] \geq 1/q'(\lambda)$, we obtain $\Pr[\widehat{\text{BAD}}] = p \geq (1/q'(\lambda) - \text{negl}(\lambda))/2$. $\square$

We proceed to case analysis depending on which trapdoor witness is used to complete SWIAOK of the left interaction in the main thread. For each $\beta \in \{0, 1\}$, let us say that event $\widehat{\text{BAD}}_\beta$ occurs if (i) $\widehat{\text{BAD}}$ occurs and (ii) SWIAOK of the left interaction is completed using a witness of form $w_{\text{td}} = (\beta, *)$ in the main thread. Similarly, let us say that event $\widehat{\text{BAD}}_2$ occurs if (i) $\widehat{\text{BAD}}$ occurs and (ii) the left interaction is aborted in the main thread. Given Claim 5, there exists $k^* \in \{0, 1, 2\}$ such that $\widehat{\text{BAD}}_{k^*}$ occurs with probability at least $1/3\hat{q}(\lambda)$. In the following, we focus on the case $k^* \in \{0, 1\}$. (The case of $k^* = 2$ can be handled analogously.)

Next, we design a series of hybrid experiments in which $\mathcal{SE}'$ is gradually modified, and we show that $\widehat{\text{BAD}}_{k^*}$ occurs with non-negligible probability in each hybrid.

- Hybrid H_0 is the experiment where $\mathcal{SE}'(1^\lambda, z)$ is executed. From the observation above, $\widehat{\text{BAD}}_{k^*}$ occurs with probability at least $\varepsilon_0 := 1/3\hat{q}(\lambda)$ in H_0 .
- Hybrid H_1 is identical to H_0 except that in the main thread, the WIPOK response message $\tilde{w}_3$ in the right interaction is computed by $\tilde{w}_3 \leftarrow \text{WIPOK}.\text{P}_2(\tilde{\text{st}}_{\text{wi}}, \tilde{x}_{V,0}, \tilde{x}_{V,1}, \tilde{w}_{V,\tilde{\beta}}, \tilde{w}_2)$ using the real prover algorithm (rather than by $\tilde{w}_3 \leftarrow \text{WIPOK}.\hat{\text{P}}_2(\tilde{\text{st}}_{\text{wi}}, \tilde{\text{aux}}_{\text{wi}}, \tilde{x}_{V,0}, \tilde{x}_{V,1}, \tilde{w}_{V,0}, \tilde{w}_{V,1}, \tilde{w}_2)$ using the hybrid prover algorithm).²⁸
- Hybrid H_2 is identical to H_1 except in how the trapdoor witness w_{td} is computed in the simulation part. Concretely, when the Round-3 message msg_3 is accepting in the left interaction in the main thread, the left look-ahead threads are no longer generated (and therefore $\text{WIPOK}.\text{Ext}$ is no longer executed either), and w_{td} is computed by brute force as follows.

1. Extract the committed values of the k^* -th outer NMC commitment by brute force. If the k^* -th outer NMC commitment is invalid (i.e., cannot be decommitted to any value), output fail and abort. Otherwise, let $i\text{-dec}$ denote the extracted value.

²⁶To see that the left rewinding in $\mathcal{SE}'$ does not interfere with the adaptive-input computational special soundness of SWIAOK, observe that the left rewinding can only require sampling a new challenge message for SWIAOK (which can be done without knowing the internal state of the SWIAOK verifier since SWIAOK is a public-coin protocol).

²⁷We require the rewinding version of the adaptive-input OR-indistinguishability since the left rewinding can require sampling new response messages for WIPOK. To see that the right rewinding is not problematic, observe that the right look-ahead thread reuses the same WIPOK response message as the main thread.

²⁸We emphasize that the switch to the real prover algorithm is made only in the main thread. In the left look-ahead threads, the WIPOK response message in the right interaction is computed as in H_1 , i.e., using the hybrid prover algorithm.

2. Let $w_{\text{td}} := (k^*, \text{i-dec})$.

- Hybrid H_3 is identical to H_2 except that in the main thread, the $(1 - \tilde{\beta})$ -th outer NMC commitment in the right interaction is computed by committing to a random string (i.e., by running $\text{NMC.S}_1(1^\lambda, \widetilde{\text{tag}}, r')$ for random $r' \in \{0, 1\}^\lambda$ rather than $\text{NMC.S}_1(1^\lambda, \widetilde{\text{tag}}, \widetilde{\text{i-dec}_{1-\beta}})$ in Round 1 of the right interaction in the main thread).

Claim 6. $\widehat{\text{BAD}}_{k^*}$ occurs in H_1 with probability at least $\varepsilon_1 := (\varepsilon_0/2)^3 - \text{negl}(\lambda)$.

Proof. Recall that H_1 differs from H_0 in that the real prover algorithm of WIPOK is used in Round 3 of the right interaction in the main thread (while the hybrid prover algorithm remains to be used in the left look-ahead threads). We show this claim by relying on Lemma 1 (the property shown for WIPOK in Section 4).

Informally, we use Lemma 1 as follows. Recall that Lemma 1 is stated about three Turing machines, one is executed to obtain intermediate information $\text{st}_{\mathcal{A}}$ and the remaining two are executed independently of each other on input $\text{st}_{\mathcal{A}}$. Accordingly, we consider decomposing the execution of H_0 (resp., H_1) into three components.

- The first component runs the main thread until the point just before completing Round 3 of the right interaction (more concretely, just before obtaining the WIPOK response message $\widetilde{\text{wi}}_3$ in the right interaction).
- Given the partial transcript generated by the first component, the second component obtains a trapdoor witness w_{td} by generating two left look-ahead threads and invoking WIPOK.Ext .
- Given the partial transcript generated by the first component and the trapdoor witness w_{td} generated by the second component, the third component runs the rest of the main thread (starting from the generation of the WIPOK response message $\widetilde{\text{wi}}_3$ in the right interaction) and the subsequent extraction part of $\mathcal{SE}'$.

Note that H_0 and H_1 differ only in the third component, where a message obtained by the hybrid prover of WIPOK is used in H_0 while a message obtained by the real prover is used in H_1 . We cannot use Lemma 1 yet since the second and third components are not independent (the third component receives the trapdoor witness w_{td} from the second component). Toward using Lemma 1, we observe that since we have $\Pr[\widehat{\text{BAD}}_{k^*}] \geq \varepsilon_0$ in H_0 , the following occur with probability at least ε_0 when H_0 is decomposed as above.

- In the first component, the k^* -th inner NMC commit message $\text{i-nm}_{k^*,1}$ in Round 1 of the left interaction uniquely determines a decommitment candidate i-dec satisfying $\text{NMC.RD}_{\text{weak}}(\text{i-nm}_{k^*,1}, \text{i-dec}) = 1$.
- In the second component, a valid trapdoor witness w_{td} of the form $(k^*, *)$ is obtained, and furthermore, it satisfies $w_{\text{td}} = (k^*, \text{i-dec})$ for i-dec determined in the first component.²⁹
- In the third component, after the execution of $\mathcal{SE}'$ is completed by using $w_{\text{td}} = (k^*, \text{i-dec})$ for SWIAOK in the left interaction, $\widehat{\text{BAD}}_{k^*}$ occurs in the emulated execution of $\mathcal{SE}'$.

Now, observe that the above remains to hold even when the third component is executed independently of the second one as follows: instead of having it use the trapdoor witness that is extracted in the second component, have it use the trapdoor witness that is defined based on the decommitment candidate that is determined in the first component. Now that the second and the third components become independent, we can use Lemma 1 to show that the above occurs with non-negligible probability even in H_1 , which implies that $\widehat{\text{BAD}}_{k^*}$ occurs in H_1 with non-negligible probability.

Formally, to use Lemma 1, we consider the following Turing machines.

- $\mathcal{A}_1(1^\lambda, \text{wi}_1, z)$ internally invokes $\mathcal{SE}'$ and emulates H_0 with the following differences.

²⁹To see the furthermore part, observe that by the uniqueness of valid decommitment candidates, a valid trapdoor witness of the form $(k^*, *)$ must be equal to $(k^*, \text{i-dec})$.

- In the main thread, wi_1 is forwarded as the WIPOK commit message in Round 1 of the right interaction.
- In the main thread, when inner NMC commit messages $\{i\text{-nm}_{b,1}\}_{b \in \{0,1\}}$ are obtained in Round 1 of the left interaction, a decommitment candidate $i\text{-dec}$ such that $\text{NMC.R}_{D,\text{weak}}(i\text{-nm}_{k^*,1}, i\text{-dec}) = 1$ is obtained by brute force.
- The emulation of H_0 is suspended at the point where the WIPOK response message is to be computed in Round 3 of the right interaction in the main thread. Let $\tilde{x}_{V,0}, \tilde{x}_{V,1}, \tilde{w}_{V,0}, \tilde{w}_{V,1}, \tilde{wi}_2$ be the WIPOK statements etc. that are to be used to generate the WIPOK response message, and let $\text{st}_{\mathcal{SE}}$ be the current state of $\mathcal{SE}'$.

Then, $\mathcal{A}_1$ lets $\text{st}_{\mathcal{A}} := (\text{st}_{\mathcal{SE}}, i\text{-dec})$ and outputs $(\tilde{x}_{V,0}, \tilde{x}_{V,1}, \tilde{w}_{V,0}, \tilde{w}_{V,1}, \tilde{wi}_2, \text{st}_{\mathcal{A}})$.

- $\hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}})$ parses $\text{st}_{\mathcal{A}}$ as $\text{st}_{\mathcal{A}} := (\text{st}_{\mathcal{SE}}, i\text{-dec})$ and continues the emulation of H_0 as follows.
 1. Using $\text{st}_{\mathcal{SE}}$, reconstruct the main thread until the point just before Round 2 of the left interaction.
 2. Obtain a trapdoor witness w_{td} by generating two left look-ahead threads and running WIPOK.Ext as in H_0 , where the oracle $\mathcal{O}$ is used whenever a new WIPOK response message is required in the left look-ahead threads.

Then, $\hat{\mathcal{A}}_2$ outputs 1 if and only if the trapdoor witness w_{td} satisfies $w_{\text{td}} = (k^*, i\text{-dec})$.

- $\mathcal{A}_2(\text{st}_{\mathcal{A}}, wi_3)$ parses $\text{st}_{\mathcal{A}}$ as $\text{st}_{\mathcal{A}} := (\text{st}_{\mathcal{SE}}, i\text{-dec})$ and continues the emulation of H_0 as follows.
 1. Using $\text{st}_{\mathcal{SE}}$, reconstruct the main thread until the point at which the WIPOK response message is to be computed in Round 3 of the right interaction. Then, complete Round 3 of the right interaction by using wi_3 as the WIPOK response message.
 2. Emulate the rest of H_0 honestly except that in the main thread, if the Round-3 message msg_3 in the left interaction is accepting, then Round 4 is completed by using $(k^*, i\text{-dec})$ as the witness for SWIAOK (without generating left look-ahead threads).

Then, $\mathcal{A}_2$ outputs 1 if and only if $\widehat{\text{BAD}}_{k^*}$ occurs.

Note that when wi_3 is computed by the hybrid prover of WIPOK, the adversary $(\mathcal{A}_1, \hat{\mathcal{A}}_2, \mathcal{A}_2)$ perfectly emulates H_0 when a trapdoor witness of the form $(k^*, *)$ is obtained in the left interaction. Thus, we have

$$\Pr \left[\begin{array}{l} \mathcal{A}_2(\text{st}_{\mathcal{A}}, wi_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}}) = 1 \end{array} \middle| \begin{array}{l} (wi_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, wi_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, wi_1, z) \\ wi_3 \leftarrow \text{WIPOK.P}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, wi_2) \end{array} \right] \geq \varepsilon_0.$$

Thus, we can use [Lemma 1](#) to obtain

$$\Pr \left[\begin{array}{l} \mathcal{A}_2(\text{st}_{\mathcal{A}}, wi_3) = 1 \\ \wedge \hat{\mathcal{A}}_2^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(\text{st}_{\mathcal{A}}) = 1 \end{array} \middle| \begin{array}{l} (wi_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.P}_1(1^\lambda, \beta) \\ (x_0, x_1, w_0, w_1, wi_2, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(1^\lambda, wi_1, z) \\ \textcolor{violet}{wi_3} \leftarrow \textcolor{violet}{\text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, wi_2)} \end{array} \right] \geq \left(\frac{\varepsilon_0}{2}\right)^3 - \text{negl}(\lambda). \quad (13)$$

Note that when wi_3 is computed by the real prover of WIPOK, the adversary $(\mathcal{A}_1, \hat{\mathcal{A}}_2, \mathcal{A}_2)$ perfectly emulates H_1 when a trapdoor witness of the form $(k^*, *)$ is obtained in the left interaction. Thus, from (13), $\widehat{\text{BAD}}_{k^*}$ occurs in H_1 with probability at least $\varepsilon_1 := (\varepsilon_0/2)^3 - \text{negl}(\lambda)$. $\square$

Claim 7. $\widehat{\text{BAD}}_{k^*}$ occurs in H_2 with probability at least $\varepsilon_2 := \varepsilon_1$.

Proof. By definition, when msg_3 is rejecting, H_2 is identical to H_1 . We therefore focus on the case that msg_3 is accepting, and we show that when $\widehat{\text{BAD}}_{k^*}$ occurs in H_1 , the same trapdoor witness w_{td} is used in H_1 and H_2 . Suppose $\widehat{\text{BAD}}_{k^*}$ occurs in H_1 . Let $w_{\text{td}}^* = (k^*, \text{i-dec}^*)$ denote the witness used in H_1 in Round 4 of the left interaction in the main thread. By the way in which w_{td}^* is defined, i-dec^* is guaranteed to be the (unique) committed value of the k^* -th outer NMC commitment. Thus, when the committed value of the k^* -th outer NMC commitment is extracted by brute force, the extracted value agrees with i-dec^* . Thus, even when the trapdoor witness w_{td}^* is computed by brute force as in H_2 , event $\widehat{\text{BAD}}_{k^*}$ occurs with probability at least ε_1 . $\square$

Claim 8. $\widehat{\text{BAD}}_{k^*}$ occurs in H_3 with probability at least $\varepsilon_3 := \varepsilon_4 - \text{negl}(\lambda)$.

Proof. This claim follows directly from the synchronous non-malleability of NMC since the brute-force extraction can be efficiently emulated in the non-malleability security experiment. (To see that the left rewinding is not problematic, observe that the left look-ahead threads are no longer generated in H_2 and H_3 . To see that the right rewinding is not problematic, observe that the right look-ahead thread reuses the same NMC response message as the main thread.) $\square$

Now, we derive a contradiction by breaking the non-malleability of NMC using H_3 . Toward this end, it suffices to consider a man-in-the-middle adversary that internally emulates H_3 while (i) using the external committer to generate the $(1 - \widetilde{\beta})$ -th inner NMC commitment in the right interaction and (ii) using the external receiver for the k^* -th outer NMC commitment in the left interaction. Since this adversary perfectly emulates the experiment H_3 , event $\widehat{\text{BAD}}_{k^*}$ occurs in the internally emulated experiment H_3 with non-negligible probability. Furthermore, when event $\widehat{\text{BAD}}_{k^*}$ occurs in the internally emulated experiment H_3 , the adversary can obtain the value that the external committer committed to. Thus, the adversary can trivially break the non-malleability of NMC.³⁰

5.6.2 Asynchronous Case ($j^* \in \{2, 3, 4\}$)

The case of $j^* = 2$ can be analyzed in a similar (and somewhat simpler) way to the synchronous case. In particular, it suffices to make the following modifications.

1. Hybrid simulator-extractor $\mathcal{SE}_1$ outputs fail and aborts if the i^* -th right interaction is not scheduled in Schedule 2 (rather than Schedule 1) in the main thread.
2. [Claim 5](#) is proved by relying on the non-rewinding version of the adaptive-input OR-indistinguishability of the hybrid prover of WIPOK.³¹
3. Hybrids $H_1, \dots, H_3$ are replaced with the following hybrids H'_1, H'_2 .
 - Hybrid H'_1 is identical to H_0 except that in the right interaction in the main thread, the WIPOK messages w_{i_1} and w_{i_3} are computed by using the real prover algorithms.
 - Hybrid H'_2 is identical to H'_1 except that in the right interaction in the main thread, the $(1 - \widetilde{\beta})$ -th outer NMC commitment is computed by committing to a random string of an appropriate length.

The hybrids H'_1, H'_2 are analyzed as in the proof of soundness ([Section 5.2](#)) using the indistinguishability of the real and hybrid provers of WIPOK and the hiding property of NMC.

³⁰Note that the $(1 - \widetilde{\beta})$ -th inner NMC commitment in the right interaction only needs to be non-malleable with the b^* -th outer NMC in the left interaction, and it does not need to be non-malleable with the inner NMC commitments in the left interaction. Because of this asymmetry, the inner commitments actually do not need to satisfy full-fledged non-malleability. (For example, if the outer NMC commitments satisfy a sufficiently strong extractability property, we can use any non-interactive commitment scheme for the inner commitments.) We chose to use NMC for the inner commitment for simplicity.

³¹The non-rewinding version is sufficient since in Schedule 2, the left rewinding can only require sampling a new set of commit, challenge, and response messages of WIPOK (which can be done by anyone using fresh randomness).

The case of $j^* \in \{3, 4\}$ can be analyzed in a similar (and much simpler) way. The key point is that in this case, either the right rewinding can only require re-executing the left interaction from the beginning or it does not require any new message in the left interaction at all. Specifically, we replace hybrid simulator-extractor $\mathcal{SE}'$ with a one where (i) the execution is aborted if the i^* -th right interaction is not scheduled in Schedule 3 or 4, (ii) the witness extraction is only done from the i^* -th right interaction, (iii) only a single right look-ahead thread is generated during the right rewinding, and (iv) the prover messages in the left interaction are generated using the honest prover algorithm (with the help of the witness-selecting machine). As before, we can show that event BAD_{i^*, j^*} occurs in this simulator-extractor with non-negligible probability. Then, since there is no left look-ahead thread in this simulator-extractor, we can derive a contradiction as in the proof of soundness (Section 5.2).

References

- [Bar02] Boaz Barak. Constant-round coin-tossing with a man in the middle or realizing the shared random string model. In *43rd FOCS*, pages 345–355. IEEE Computer Society Press, November 2002.
- [BCO⁺24] Vincenzo Botta, Michele Ciampi, Emmanuela Orsini, Luisa Siniscalchi, and Ivan Visconti. Black-box (and fast) non-malleable zero knowledge. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 458–490. Springer, Cham, August 2024.
- [Blu82] Manuel Blum. Coin flipping by telephone. In *Proc. IEEE Spring COMPCOM*, pages 133–137, 1982.
- [BPS06] Boaz Barak, Manoj Prabhakaran, and Amit Sahai. Concurrent non-malleable zero knowledge. In *47th FOCS*, pages 345–354. IEEE Computer Society Press, October 2006.
- [CLOS02] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. Universally composable two-party and multi-party secure computation. In *34th ACM STOC*, pages 494–503. ACM Press, May 2002.
- [CLP13] Kai-Min Chung, Huijia Lin, and Rafael Pass. Constant-round concurrent zero knowledge from P-certificates. In *54th FOCS*, pages 50–59. IEEE Computer Society Press, October 2013.
- [CLP15] Kai-Min Chung, Huijia Lin, and Rafael Pass. Constant-round concurrent zero-knowledge from indistinguishability obfuscation. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 287–307. Springer, Berlin, Heidelberg, August 2015.
- [CLP20] Rohit Chatterjee, Xiao Liang, and Omkant Pandey. Improved black-box constructions of composable secure computation. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *ICALP 2020*, volume 168 of *LIPICs*, pages 28:1–28:20. Schloss Dagstuhl, July 2020.
- [COS22] Michele Ciampi, Emmanuela Orsini, and Luisa Siniscalchi. Four-round black-box non-malleable schemes from one-way permutations. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 300–329. Springer, Cham, November 2022.
- [COSV17a] Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Ivan Visconti. Delayed-input non-malleable zero knowledge and multi-party coin tossing in four rounds. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 711–742. Springer, Cham, November 2017.
- [COSV17b] Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Ivan Visconti. Delayed-input non-malleable zero knowledge and multi-party coin tossing in four rounds. Cryptology ePrint Archive, Report 2017/931, Version 20181022:155022, 2017. A preprint version of [COSV17a].

- [COSV17c] Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Ivan Visconti. Four-round concurrent non-malleable commitments from one-way functions. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 127–157. Springer, Cham, August 2017.
- [CPS⁺16a] Michele Ciampi, Giuseppe Persiano, Alessandra Scafuro, Luisa Siniscalchi, and Ivan Visconti. Improved OR-composition of sigma-protocols. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part II*, volume 9563 of *LNCS*, pages 112–141. Springer, Berlin, Heidelberg, January 2016.
- [CPS⁺16b] Michele Ciampi, Giuseppe Persiano, Alessandra Scafuro, Luisa Siniscalchi, and Ivan Visconti. Online/offline OR composition of sigma protocols. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 63–92. Springer, Berlin, Heidelberg, May 2016.
- [DDN00] Danny Dolev, Cynthia Dwork, and Moni Naor. Nonmalleable cryptography. *SIAM Journal on Computing*, 30(2):391–437, 2000.
- [DPP97] Ivan Damgård, Torben P. Pedersen, and Birgit Pfitzmann. On the existence of statistically hiding bit commitment schemes and fail-stop signatures. *Journal of Cryptology*, 10(3):163–194, June 1997.
- [FKP19] Cody Freitag, Ilan Komargodski, and Rafael Pass. Non-uniformly sound certificates with applications to concurrent zero-knowledge. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 98–127. Springer, Cham, August 2019.
- [FS90a] Uriel Feige and Adi Shamir. Witness indistinguishable and witness hiding protocols. In *22nd ACM STOC*, pages 416–426. ACM Press, May 1990.
- [FS90b] Uriel Feige and Adi Shamir. Zero knowledge proofs of knowledge in two rounds. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 526–544. Springer, New York, August 1990.
- [GGJS12] Sanjam Garg, Vipul Goyal, Abhishek Jain, and Amit Sahai. Concurrently secure computation in constant rounds. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 99–116. Springer, Berlin, Heidelberg, April 2012.
- [GK96] Oded Goldreich and Hugo Krawczyk. On the composition of zero-knowledge proof systems. *SIAM Journal on Computing*, 25(1):169–192, 1996.
- [Gol01] Oded Goldreich. *Foundations of Cryptography: Basic Tools*, volume 1. Cambridge University Press, Cambridge, UK, 2001.
- [Goy11] Vipul Goyal. Constant round non-malleable protocols using one way functions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 695–704. ACM Press, June 2011.
- [GPR15] Vipul Goyal, Omkant Pandey, and Silas Richelson. Textbook non-malleable commitments. Cryptology ePrint Archive, Report 2015/1178, Version 20170521:195131, 2015. A preprint version of [GPR16].
- [GPR16] Vipul Goyal, Omkant Pandey, and Silas Richelson. Textbook non-malleable commitments. In Daniel Wichs and Yishay Mansour, editors, *48th ACM STOC*, pages 1128–1141. ACM Press, June 2016.

- [GR19] Vipul Goyal and Silas Richelson. Non-malleable commitments using Goldreich-Levin list decoding. In David Zuckerman, editor, *60th FOCS*, pages 686–699. IEEE Computer Society Press, November 2019.
- [GRRV21] Vipul Goyal, Silas Richelson, Alon Rosen, and Margarita Vald. An algebraic approach to nonmalleability. *SIAM Journal on Computing*, 50(5):1537–1579, 2021.
- [HM96] Shai Halevi and Silvio Micali. Practical and provably-secure commitment schemes from collision-free hashing. In Neal Koblitz, editor, *CRYPTO’96*, volume 1109 of *LNCS*, pages 201–215. Springer, Berlin, Heidelberg, August 1996.
- [JP14] Abhishek Jain and Omkant Pandey. Non-malleable zero knowledge: Black-box constructions and definitional relationships. In Michel Abdalla and Roberto De Prisco, editors, *SCN 14*, volume 8642 of *LNCS*, pages 435–454. Springer, Cham, September 2014.
- [Kiy20] Susumu Kiyoshima. Statistical concurrent non-malleable zero-knowledge from one-way functions. *Journal of Cryptology*, 33(3):1318–1361, July 2020.
- [KLP22] Allen Kim, Xiao Liang, and Omkant Pandey. A new approach to efficient non-malleable zero-knowledge. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 389–418. Springer, Cham, August 2022.
- [LP11a] Huijia Lin and Rafael Pass. Concurrent non-malleable zero knowledge with adaptive inputs. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 274–292. Springer, Berlin, Heidelberg, March 2011.
- [LP11b] Huijia Lin and Rafael Pass. Constant-round non-malleable commitments from any one-way function. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 705–714. ACM Press, June 2011.
- [LP15] Huijia Lin and Rafael Pass. Constant-round nonmalleable commitments from any one-way function. *Journal of the ACM*, 62(1), March 2015.
- [LPTV10] Huijia Lin, Rafael Pass, Wei-Lung Dustin Tseng, and Muthuramakrishnan Venkatasubramanian. Concurrent non-malleable zero knowledge proofs. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 429–446. Springer, Berlin, Heidelberg, August 2010.
- [LPV08] Huijia Lin, Rafael Pass, and Muthuramakrishnan Venkatasubramanian. Concurrent non-malleable commitments from any one-way function. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 571–588. Springer, Berlin, Heidelberg, March 2008.
- [LS91] Dror Lapidot and Adi Shamir. Publicly verifiable non-interactive zero-knowledge proofs. In Alfred J. Menezes and Scott A. Vanstone, editors, *CRYPTO’90*, volume 537 of *LNCS*, pages 353–365. Springer, Berlin, Heidelberg, August 1991.
- [MU10] Jörn Müller-Quade and Dominique Unruh. Long-term security and universal composability. *Journal of Cryptology*, 23(4):594–671, October 2010.
- [OOR⁺14] Claudio Orlandi, Rafail Ostrovsky, Vanishree Rao, Amit Sahai, and Ivan Visconti. Statistical concurrent non-malleable zero knowledge. In Yehuda Lindell, editor, *TCC 2014*, volume 8349 of *LNCS*, pages 167–191. Springer, Berlin, Heidelberg, February 2014.
- [OPV10] Rafail Ostrovsky, Omkant Pandey, and Ivan Visconti. Efficiency preserving transformations for concurrent non-malleable zero knowledge. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 535–552. Springer, Berlin, Heidelberg, February 2010.

- [Pas11] Rafael Pass. Limits of provable security from standard assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 109–118. ACM Press, June 2011.
- [PPS15] Omkant Pandey, Manoj Prabhakaran, and Amit Sahai. Obfuscation-based non-black-box simulation and four message concurrent zero knowledge for NP. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 638–667. Springer, Berlin, Heidelberg, March 2015.
- [PR08a] Rafael Pass and Alon Rosen. Concurrent nonmalleable commitments. *SIAM Journal on Computing*, 37(6):1891–1925, 2008.
- [PR08b] Rafael Pass and Alon Rosen. New and improved constructions of nonmalleable cryptographic protocols. *SIAM Journal on Computing*, 38(2):702–752, 2008.
- [Ven14] Muthuramakrishnan Venkitasubramaniam. On adaptively secure protocols. In Michel Abdalla and Roberto De Prisco, editors, *SCN 14*, volume 8642 of *LNCS*, pages 455–475. Springer, Cham, September 2014.

A Proof of Theorem 3

A.1 Preliminaries

We start by recalling the definitions of several basic primitives.

A.1.1 Non-interactive Commitments

Definition 10 (Non-interactive commitment). *A pair of Turing machines $\text{Com} = (\text{Com.S}, \text{Com.R}_D)$, where Com.S is probabilistic and Com.R_D is deterministic, is called a non-interactive commitment scheme if it satisfies the following.*

Correctness. *For every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$,*

$$\Pr \left[\text{Com.R}_D(1^\lambda, \text{com}, b, \text{dec}) = 1 \mid (\text{com}, \text{dec}) \leftarrow \text{Com.S}(1^\lambda, b) \right] = 1.$$

Perfect binding. *Fix any $\lambda \in \mathbb{N}$, and let $C_b := \{\text{com} \mid \exists \text{dec s.t. } \text{Com.R}_D(1^\lambda, \text{com}, b, \text{dec}) = 1\}$ for each $b \in \{0, 1\}$. Then, $C_0 \cap C_1 = \emptyset$.*

Computational hiding *For every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$, let $\text{com}_{\lambda,b}$ be the random variable representing the first output of $\text{Com.S}(1^\lambda, b)$. Then, the ensembles $\{\text{com}_{\lambda,0}\}_{\lambda \in \mathbb{N}}$ and $\{\text{com}_{\lambda,1}\}_{\lambda \in \mathbb{N}}$ are computationally indistinguishable.*

It is known that a non-interactive commitment scheme can be constructed from any injective one-way function [Blu82].

A.1.2 Non-interactive Instance-dependent Trapdoor Commitment

Definition 11 (Non-interactive instant-dependent trapdoor commitment). *Let L be an NP language. A pair of Turing machines $\text{TC} = (\text{TC.S}, \text{TC.R}_D, \text{TC.FakeS}, \text{TC.FakeOpen})$, where TC.S and TC.FakeS are probabilistic and TC.R_D and TC.FakeOpen are deterministic, is called a non-interactive instance-dependent trapdoor commitment scheme for L if it satisfies the following.*

Correctness. *For every $\lambda \in \mathbb{N}$, $x \in \{0, 1\}^*$, and $m \in \{0, 1\}^*$,*

$$\Pr \left[\text{TC.R}_D(1^\lambda, x, \text{tcom}, m, \text{tdec}) = 1 \mid (\text{tcom}, \text{tdec}) \leftarrow \text{TC.S}(1^\lambda, x, m) \right] = 1.$$

Perfect binding. Fix any $\lambda \in \mathbb{N}$, $x \notin \mathbf{L}$, $m_0 \in \{0,1\}^*$, and $m_1 \in \{0,1\}^*$ such that $m_0 \neq m_1$. Let $C_b := \{\text{tcom} \mid \exists \text{tdec s.t. } \text{TC.RD}(1^\lambda, x, \text{tcom}, m_b, \text{tdec}) = 1\}$ for each $b \in \{0,1\}$. Then, $C_0 \cap C_1 = \emptyset$.

Equivocality correctness. For every $\lambda \in \mathbb{N}$, $(x, w) \in \mathbf{R}_L$, and $m \in \{0,1\}^*$,

$$\Pr \left[\text{TC.RD}(1^\lambda, x, \text{tcom}, m, \text{tdec}) = 1 \mid \begin{array}{l} (\text{tcom}, \text{st}) \leftarrow \text{TC.FakeS}(1^\lambda, x) \\ \text{tdec} := \text{TC.FakeOpen}(\text{st}, w, m) \end{array} \right] = 1.$$

Equivocality hiding. For every pair of PPT Turing machines $(\mathcal{V}_1, \mathcal{V}_2)$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ and $z \in \{0,1\}^*$, the random variable out satisfies $\Pr[\text{out} = 1] \leq 1/2 + \text{negl}(\lambda)$ in the following experiment.

1. Compute $(x, w, m) \leftarrow \mathcal{V}_1(1^\lambda, z)$. If $(x, w) \notin \mathbf{R}_L$, abort.
2. Sample a random bit $\text{ch} \in \{0,1\}$. If $\text{ch} = 0$, compute $(\text{tcom}, \text{tdec})$ by $(\text{tcom}, \text{tdec}) \leftarrow \text{TC.S}(1^\lambda, x, m)$. If $\text{ch} = 1$, compute $(\text{tcom}, \text{tdec})$ by $(\text{tcom}, \text{st}) \leftarrow \text{TC.FakeS}(1^\lambda, x)$ and $\text{tdec} := \text{TC.FakeOpen}(\text{st}, w, m)$.
3. Compute $\tilde{\text{ch}} \leftarrow \mathcal{V}_2(\text{st}_V, \text{tcom}, \text{tdec})$ and let $\text{out} := 1$ if and only if $\tilde{\text{ch}} = \text{ch}$.

It is known that a non-interactive instance-dependent trapdoor commitment scheme can be constructed for an NP-complete language (Graph Hamiltonicity) from any non-interactive commitment scheme [FS90b, CLOS02].

A.1.3 3-round Adaptive-input Special-sound Special-HVZK Proof

Definition 12 (3-round adaptive-input special-sound special-HVZK proof). Let $\mathbf{L}$ be an NP language. A tuple of PPT Turing machines (P_1, P_2, V, V_D) is called a 3-round adaptive-input special-sound special honest-verifier zero-knowledge (special-HVZK) proof if it satisfies the following.

Delayed-input correctness. For every $(x, w) \in \mathbf{R}_L$,

$$\Pr \left[V_D(x, \tau) = 1 \mid \begin{array}{l} (\text{msg}_1, \text{st}) \leftarrow P_1(1^{|x|}) \\ \text{msg}_2 \leftarrow V(1^{|x|}) \\ \text{msg}_3 \leftarrow P_2(\text{st}, x, w, \text{msg}_2) \\ \tau := (\text{msg}_1, \text{msg}_2, \text{msg}_3) \end{array} \right] = 1.$$

Delayed-input special soundness. There exists a deterministic polynomial-time Turing machine Ext (called the extractor) such that the following holds.

- Fix any $x_0 \in \{0,1\}^*$, $x_1 \in \{0,1\}^*$, $\tau_0 := (\text{msg}_{0,1}, \text{msg}_{0,2}, \text{msg}_{0,3}) \in (\{0,1\}^*)^3$, and $\tau_1 := (\text{msg}_{1,1}, \text{msg}_{1,2}, \text{msg}_{1,3}) \in (\{0,1\}^*)^3$. Suppose it holds $\text{msg}_{0,1} = \text{msg}_{1,1}$, $\text{msg}_{0,2} \neq \text{msg}_{1,2}$, and $V_D(x_b, \tau_b) = 1$ for every $b \in \{0,1\}$. Then, $\text{Ext}(x_0, \tau_0, x_1, \tau_1)$ outputs a value w_0 such that $(x_0, w_0) \in \mathbf{R}_L$.

Delayed-input Special HVZK. There exists a PPT Turing machine Sim (called the simulator) such that for every sequence $\{w_x, \text{msg}_{x,2}\}_{x \in \mathbf{L}}$ such that $w_x \in \mathbf{R}_L(x)$ and $\text{msg}_{x,2} \in \{0,1\}^*$, the following two ensembles are computationally indistinguishable.

- $\{(\text{msg}_1, \text{msg}_{x,2}, \text{msg}_3) \mid (\text{msg}_1, \text{st}) \leftarrow P_1(1^{|x|}); \text{msg}_3 \leftarrow P_2(\text{st}, x, w, \text{msg}_{x,2})\}_{x \in \mathbf{L}}$.
- $\{(\text{msg}_1, \text{msg}_{x,2}, \text{msg}_3) \mid (\text{msg}_1, \text{msg}_3) \leftarrow \text{Sim}(x, \text{msg}_{x,2})\}_{x \in \mathbf{L}}$.

It is known that a 3-round adaptive-input special-sound special-HVZK proof can be constructed from any non-interactive commitment scheme. Specifically, it suffices to augment the delayed-input proof of knowledge of Lapidot and Shamir [LS91] with a trick by Ciampi et al. [COSV17b, Appendix C.1] to obtain delayed-input special soundness. The resultant scheme has a public-coin verifier.

A.2 Protocol Description

Fix any NP languages $\mathbf{L}$. We use the following building blocks.

- A non-interactive perfectly binding commitment scheme $\text{Com} = (\text{Com.S}, \text{Com.R}_D)$.
- A non-interactive instance-dependent trapdoor commitment scheme $\text{TC} = (\text{TC.S}, \text{TC.R}_D, \text{TC.FakeS}, \text{TC.FakeOpen})$ for the language $\mathbf{L}_{\text{tcom}} := \{(\lambda, \text{com}, b) \mid \exists \text{dec s.t. } \text{Com.R}_D(1^\lambda, \text{com}, b, \text{dec}) = 1\}$.
- A 3-round adaptive-input special-sound special-HVZK argument $\text{LS} = (\text{LS.P}_1, \text{LS.P}_2, \text{LS.V}, \text{LS.V}_D)$ for $\mathbf{L}$. Let LS.Sim be the simulator for the special HVZK property and LS.Ext be the extractor for the delayed-input special soundness.

At a high level, the protocol proceeds as follows. To prove a statement $x_0 \in \mathbf{L} \vee x_1 \in \mathbf{L}$, the prover generates two proofs by using LS , one for statement $x_0 \in \mathbf{L}$ and the other for statement $x_1 \in \mathbf{L}$. To complete these two proofs while only having a witness for $x_\beta \in \mathbf{L}$, the prover avoids sending messages in the clear in the first round. Instead, the prover commits to the messages in the first round and reveals them in the last round after receiving challenge messages from the verifier. (In the last round, the prover also sends the response messages to complete the two proofs.) The key point is that the prover makes the commitments by using TC in a way that the commitment is binding in the proof for $x_\beta \in \mathbf{L}$ and is equivocal in the proof for $x_{1-\beta} \in \mathbf{L}$. On the one hand, the prover can complete the proof for $x_\beta \in \mathbf{L}$ since the prover has a valid witness for the statement. On the other hand, the prover can complete the proof for $x_{1-\beta} \in \mathbf{L}$ by using the SHVZK simulator of LS . Specifically, for the latter, the prover generates an accepting transcript by using the SHVZK simulator and opens the commitments accordingly by using the equivocality of TC .

The formal description of the protocol, $\text{WIPOK} = (\text{WIPOK.P}_1, \text{WIPOK.P}_2, \text{WIPOK.V}, \text{WIPOK.V}_D)$, is given in [Figure 5](#).

A.3 Security proof

Let $(\text{WIPOK}.\hat{\text{P}}_1, \text{WIPOK}.\hat{\text{P}}_2, \text{WIPOK}.\hat{\text{P}}_{1.5})$ be defined as in [Figure 6](#). We prove each of the required properties below.

Delayed-input correctness. Delayed-input correctness can be verified by inspection.

Delayed-input special soundness. Define the function EQ and the extractor Ext as follows.

- $\text{EQ}(\text{wi}_2, \text{wi}'_2)$: Parse wi_2 as $\text{wi}_2 = \{\text{ls}_{b,2}\}_{b \in \{0,1\}}$ and wi'_2 as $\text{wi}'_2 = \{\text{ls}'_{b,2}\}_{b \in \{0,1\}}$. Then, return 1 if and only if $\exists b \in \{0,1\}$ s.t. $\text{ls}_{b,2} = \text{ls}'_{b,2}$.
- $\text{Ext}(x, \tau, x', \tau')$:

1. Parse x, τ, x', τ' as

$$\begin{aligned} x &= (x_0, x_1), \\ \tau &= (\text{wi}_1, \text{wi}_2, \text{wi}_3) = ((\text{com}, \{\text{tcom}_b\}_{b \in \{0,1\}}), \{\text{ls}_{b,2}\}_{b \in \{0,1\}}, \{\text{ls}_{b,1}, \text{ls}_{b,3}, \text{tdec}_b\}_{b \in \{0,1\}}), \\ x' &= (x'_0, x'_1), \text{ and} \\ \tau' &= (\text{wi}'_1, \text{wi}'_2, \text{wi}'_3) = ((\text{com}', \{\text{tcom}'_b\}_{b \in \{0,1\}}), \{\text{ls}'_{b,2}\}_{b \in \{0,1\}}, \{\text{ls}'_{b,1}, \text{ls}'_{b,3}, \text{tdec}'_b\}_{b \in \{0,1\}}). \end{aligned}$$

If $\text{wi}_1 \neq \text{wi}'_1$, abort.

2. Find $b^* \in \{0,1\}$ such that $\text{ls}_{b^*,1} = \text{ls}'_{b^*,1}$. If such b^* does not exist, abort.
3. Return $\text{LS.Ext}(x_{b^*}, (\text{ls}_{b^*,1}, \text{ls}_{b^*,2}, \text{ls}_{b^*,3}), x'_{b^*}, (\text{ls}'_{b^*,1}, \text{ls}'_{b^*,2}, \text{ls}'_{b^*,3}))$.

For each $b \in \{0, 1\}$, let $(x_b, w_b) \in \mathbf{R}_L$ be an instance of the NP language L and a corresponding witness. Suppose $|x_0| = |x_1|$. Let $\lambda := |x_0|$ denote the security parameter.

Round 1. $\text{WIPOK.P}_1(1^\lambda, \beta)$:

1. Compute $(\text{com}, \text{dec}) \leftarrow \text{Com}(1^\lambda, 1 - \beta)$.
2. Compute $(\text{ls}_{\beta,1}, \text{st}_{\text{ls},\beta}) \leftarrow \text{LS.P}_1(1^\lambda)$.
3. Compute $(\text{tcom}_\beta, \text{tdec}_\beta) \leftarrow \text{TC.S}(1^\lambda, x_{\text{tcom},\beta}, \text{ls}_{\beta,1})$ and $(\text{tcom}_{1-\beta}, \text{st}_{\text{tcom},1-\beta}) \leftarrow \text{TC.FakeS}(1^\lambda, x_{\text{tcom},1-\beta})$, where $x_{\text{tcom},b} := (\lambda, \text{com}, b)$ is viewed as an instance of L_{tcom} for each $b \in \{0, 1\}$ so that $x_{\text{tcom},\beta} \notin L_{\text{tcom}}$ and $x_{\text{tcom},1-\beta} \in L_{\text{tcom}}$.
4. Output $\text{wi}_1 := (\text{com}, \{\text{tcom}_b\}_{b \in \{0,1\}})$ and $\text{st} := (\beta, \text{dec}, \text{ls}_{\beta,1}, \text{st}_{\text{ls},\beta}, \text{tdec}_\beta, \text{st}_{\text{tcom},1-\beta})$.

Round 2. $\text{WIPOK.V}(1^\lambda)$:

1. $\forall b \in \{0, 1\}$: Compute $\text{ls}_{b,2} \leftarrow \text{LS.V}(1^\lambda)$.
2. Output $\text{wi}_2 := \{\text{ls}_{b,2}\}_{b \in \{0,1\}}$.

Round 3. $\text{WIPOK.P}_2(\text{st}, x_0, x_1, w_\beta, \text{wi}_2)$:

1. Compute $\text{ls}_{\beta,3} \leftarrow \text{LS.P}_2(\text{st}_{\text{ls},\beta}, x_\beta, w_\beta, \text{ls}_{\beta,2})$.
2. Compute $(\text{ls}_{1-\beta,1}, \text{ls}_{1-\beta,3}) \leftarrow \text{LS.Sim}(x_{1-\beta}, \text{ls}_{1-\beta,2})$.
3. Compute $\text{tdec}_{1-\beta} := \text{TC.FakeOpen}(\text{st}_{\text{tcom},1-\beta}, w_{\text{tcom}}, \text{ls}_{1-\beta,1})$, where $w_{\text{tcom}} := \text{dec}$ is viewed as a witness for $x_{\text{tcom},1-\beta} \in L_{\text{tcom}}$.
4. Output $\text{wi}_3 := \{\text{ls}_{b,1}, \text{ls}_{b,3}, \text{tdec}_b\}_{b \in \{0,1\}}$.

Verification. $\text{WIPOK.V}_D(x_0, x_1, \tau)$, where $\tau = (\text{wi}_1, \text{wi}_2, \text{wi}_3)$:

1. Output 1 if and only if the following hold.
 - $\forall b \in \{0, 1\}$: $\text{TC.R}_D(1^\lambda, x_{\text{tcom},b}, \text{tcom}_b, \text{ls}_{b,1}, \text{tdec}_b) = 1$.
 - $\forall b \in \{0, 1\}$: $\text{LS.V}_D(x_b, \pi_{\text{ls},b}) = 1$, where $\pi_{\text{ls},b} := (\text{ls}_{b,1}, \text{ls}_{b,2}, \text{ls}_{b,3})$.

Figure 5: 3-round adaptive-input OR-indistinguishable special-sound proof for L .

First, we analyze EQ. Since LS has a public-coin verifier, we can assume without loss of generality that the following holds for any $\lambda \in \mathbb{N}$.³²

$$\forall b \in \{0, 1\} : \Pr \left[\text{ls}_{b,2} = \text{ls}'_{b,2} \mid \begin{array}{l} \text{ls}_{b,2} \leftarrow \text{LS.V}(1^\lambda) \\ \text{ls}'_{b,2} \leftarrow \text{LS.V}(1^\lambda) \end{array} \right] \leq \frac{1}{2^{\lambda+1}}.$$

Thus, from the union bound, we have

$$\begin{aligned} & \Pr \left[\text{EQ}(\{\text{ls}_{b,2}\}_{b \in \{0,1\}}, \{\text{ls}'_{b,2}\}_{b \in \{0,1\}}) = 1 \mid \begin{array}{l} \{\text{ls}_{b,2}\}_{b \in \{0,1\}} \leftarrow \text{WIPOK.V}(1^\lambda) \\ \{\text{ls}'_{b,2}\}_{b \in \{0,1\}} \leftarrow \text{WIPOK.V}(1^\lambda) \end{array} \right] \\ &= \Pr \left[\exists b \in \{0, 1\} \text{ s.t. } \text{ls}_{b,2} = \text{ls}'_{b,2} \mid \begin{array}{l} \{\text{ls}_{b,2}\}_{b \in \{0,1\}} \leftarrow \text{WIPOK.V}(1^\lambda) \\ \{\text{ls}'_{b,2}\}_{b \in \{0,1\}} \leftarrow \text{WIPOK.V}(1^\lambda) \end{array} \right] \\ &\leq \frac{1}{2^{\lambda+1}} + \frac{1}{2^{\lambda+1}} = \frac{1}{2^\lambda}. \end{aligned}$$

³²In particular, it suffices to assume that LS is run sufficiently many times in parallel.

Alternative Round 1. $\text{WIPOK}.\hat{P}_1(1^\lambda, \beta)$:

1. Compute $(\text{com}, \text{dec}) \leftarrow \text{Com}(1^\lambda, 1 - \beta)$.
2. $\forall b \in \{0, 1\}$: Compute $(\text{ls}_{b,1}, \text{st}_{\text{ls},b}) \leftarrow \text{LS.P}_1(1^\lambda)$.
3. Compute $(\text{tcom}_\beta, \text{tdec}_\beta) \leftarrow \text{TC.S}(1^\lambda, x_{\text{tcom},\beta}, \text{ls}_{\beta,1})$ and $(\text{tcom}_{1-\beta}, \text{st}_{\text{tcom},1-\beta}) \leftarrow \text{TC.FakeS}(1^\lambda, x_{\text{tcom},1-\beta})$, where $x_{\text{tcom},b} := (\lambda, \text{com}, b)$ is viewed as an instance of $\mathbf{L}_{\text{tcom}}$ for each $b \in \{0, 1\}$ so that $x_{\text{tcom},\beta} \notin \mathbf{L}_{\text{tcom}}$ and $x_{\text{tcom},1-\beta} \in \mathbf{L}_{\text{tcom}}$.
4. Output $\text{wi}_1 := (\text{com}, \{\text{tcom}_b\}_{b \in \{0,1\}})$, $\text{st} := (\beta, \text{dec}, \text{ls}_{\beta,1}, \text{st}_{\text{ls},\beta}, \text{tdec}_\beta, \text{st}_{\text{tcom},1-\beta})$, and $\text{aux} := (\text{ls}_{1-\beta,1}, \text{st}_{\text{ls},1-\beta})$.

Alternative Round 3. $\text{WIPOK}.\hat{P}_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$:

1. Compute $\text{ls}_{\beta,3} \leftarrow \text{LS.P}_2(\text{st}_{\text{ls},\beta}, x_\beta, w_\beta, \text{ls}_{\beta,2})$.
 2. Compute $\text{ls}_{1-\beta,3} \leftarrow \text{LS.P}_2(\text{st}_{\text{ls},1-\beta}, x_{1-\beta}, w_{1-\beta}, \text{ls}_{1-\beta,2})$.
 3. Compute $\text{tdec}_{1-\beta} := \text{TC.FakeOpen}(\text{st}_{\text{tcom},1-\beta}, w_{\text{tcom}}, \text{ls}_{1-\beta,1})$, where $w_{\text{tcom}} := \text{dec}$ is viewed as a witness for $x_{\text{tcom},1-\beta} \in \mathbf{L}_{\text{tcom}}$.
 4. Output $\text{wi}_3 := \{\text{ls}_{b,1}, \text{ls}_{b,3}, \text{tdec}_b\}_{b \in \{0,1\}}$.
-

Round 1.5. $\text{WIPOK}.\hat{P}_{1.5}(1^\lambda, \beta)$:

1. Compute $(\text{ls}_{1-\beta,1}, \text{st}_{\text{ls},1-\beta}) \leftarrow \text{LS.P}_1(1^\lambda)$.
2. Output $\text{aux} := (\text{ls}_{1-\beta,1}, \text{st}_{\text{ls},1-\beta})$.

Figure 6: Alternative prover algorithms of WIPOK.

Next, we analyze Ext. Fix any

$$\begin{aligned}
 x &= (x_0, x_1), \\
 \tau &= (\text{wi}_1, \text{wi}_2, \text{wi}_3) = ((\text{com}, \{\text{tcom}_b\}_{b \in \{0,1\}}), \{\text{ls}_{b,2}\}_{b \in \{0,1\}}, \{\text{ls}_{b,1}, \text{ls}_{b,3}, \text{tdec}_b\}_{b \in \{0,1\}}), \\
 x' &= (x'_0, x'_1), \text{ and} \\
 \tau' &= (\text{wi}'_1, \text{wi}'_2, \text{wi}'_3) = ((\text{com}', \{\text{tcom}'_b\}_{b \in \{0,1\}}), \{\text{ls}'_{b,2}\}_{b \in \{0,1\}}, \{\text{ls}'_{b,1}, \text{ls}'_{b,3}, \text{tdec}'_b\}_{b \in \{0,1\}}).
 \end{aligned}$$

Suppose we have

$$\text{wi}_1 = \text{wi}'_1, \tag{14}$$

$$\text{EQ}(\text{wi}_2, \text{wi}'_2) = 0, \text{ and} \tag{15}$$

$$\text{V}_D(x_0, x_1, \tau) = 1 \wedge \text{V}_D(x'_0, x'_1, \tau') = 1. \tag{16}$$

Let $\lambda := |x_0|$. First, we note that there always exists $b^* \in \{0, 1\}$ such that $\text{ls}_{b^*,1} = \text{ls}'_{b^*,1}$ (i.e., Ext never aborts). Indeed, the perfect binding of Com guarantees that there exists $b^* \in \{0, 1\}$ such that $(\lambda, \text{com}, b^*) \notin \mathbf{L}_{\text{tcom}}$, and for any such b^* , the perfect binding of TC implies $\text{ls}_{b^*,1} = \text{ls}'_{b^*,1}$. Next, we observe that Ext outputs a valid witness. Fix any $b^* \in \{0, 1\}$ such that $\text{ls}_{b^*,1} = \text{ls}'_{b^*,1}$. By definition, Equation (15) implies $\text{ls}_{b^*,2} \neq \text{ls}'_{b^*,2}$ and Equation (16) implies $\text{LS.V}_D(x_{b^*}, (\text{ls}_{b^*,1}, \text{ls}_{b^*,2}, \text{ls}_{b^*,3})) = 1 \wedge \text{LS.V}_D(x'_{b^*}, (\text{ls}'_{b^*,1}, \text{ls}'_{b^*,2}, \text{ls}'_{b^*,3})) = 1$. Thus, the delayed-input special soundness of LS guarantees that $\text{LS.Ext}(x_{b^*}, (\text{ls}_{b^*,1}, \text{ls}_{b^*,2}, \text{ls}_{b^*,3}), x'_{b^*}, (\text{ls}'_{b^*,1}, \text{ls}'_{b^*,2}, \text{ls}'_{b^*,3}))$ outputs a value w such that $(x_{b^*}, w) \in \mathbf{R}_L$.

Adaptive-input OR indistinguishability. Adaptive-input OR indistinguishability can be shown by combining (i) indistinguishability between the real and hybrid provers and (ii) adaptive-input OR-indistinguishability of the hybrid prover (both of which are shown below).

Delayed-input correctness of hybrid prover. Delayed-input correctness of the hybrid prover can be verified by inspection.

Indistinguishability between real and hybrid provers. The equivalence between $(\text{WIPOK.P}_1, \text{WIPOK.}\hat{\text{P}}_1)$ and $\text{WIPOK.}\hat{\text{P}}_{1.5}$ can be verified by inspection. For the indistinguishability between WIPOK.P_2 and $(\text{WIPOK.}\hat{\text{P}}_{1.5}, \text{WIPOK.}\hat{\text{P}}_2)$, observe that the only difference between the two experiments is that in the first one, $\text{ls}_{1-\beta,1}$ and $\text{ls}_{1-\beta,3}$ are generated by LS.Sim whereas in the second one, $\text{ls}_{1-\beta,1}$ and $\text{ls}_{1-\beta,3}$ are generated by LS.P_1 and LS.P_2 . Thus, the indistinguishability follows directly from the delayed-input special HVZK of LS .

Adaptive-input (rewinding) OR-indistinguishability of hybrid prover. We consider the following hybrid prover by modifying $\text{WIPOK.}\hat{\text{P}}_1$ and $\text{WIPOK.}\hat{\text{P}}_2$.

- $\text{WIPOK.}\hat{\text{P}}'_1(1^\lambda, \beta)$:
 1. Compute $(\text{com}, \text{dec}) \leftarrow \text{Com}(1^\lambda, 1 - \beta)$.
 2. $\forall b \in \{0, 1\}$: Compute $(\text{ls}_{b,1}, \text{st}_{\text{ls},b}) \leftarrow \text{LS.P}_1(1^\lambda)$.
 3. $\forall b \in \{0, 1\}$: Compute $(\text{tcom}_b, \text{tdec}_b) \leftarrow \text{TC.S}(1^\lambda, x_{\text{tcom},b}, \text{ls}_{b,1})$.
 4. Output $\text{wi}_1 := (\text{com}, \{\text{tcom}_b\}_{b \in \{0,1\}})$, $\text{st} := (\beta, \text{dec}, \text{ls}_{\beta,1}, \text{st}_{\text{ls},\beta}, \text{tdec}_\beta, \text{tdec}_{1-\beta})$, and $\text{aux} := (\text{ls}_{1-\beta,1}, \text{st}_{\text{ls},1-\beta})$.
- $\text{WIPOK.}\hat{\text{P}}'_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$:
 1. Compute $\text{ls}_{\beta,3} \leftarrow \text{LS.P}_2(\text{st}_{\text{ls},\beta}, x_\beta, w_\beta, \text{ls}_{\beta,2})$.
 2. Compute $\text{ls}_{1-\beta,3} \leftarrow \text{LS.P}_2(\text{st}_{\text{ls},1-\beta}, x_{1-\beta}, w_{1-\beta}, \text{ls}_{1-\beta,2})$.
 3. ~~Compute $\text{tdec}_{1-\beta} := \text{TC.FakeOpen}(\text{st}_{\text{tcom},1-\beta}, w_{\text{tcom}}, \text{ls}_{1-\beta,1})$, where $w_{\text{tcom}} := \text{dec}$ is viewed as a witness for $x_{\text{tcom}} \in \mathbf{L}_{\text{tcom},1-\beta}$.~~
 4. Output $\text{wi}_3 := \{\text{ls}_{b,1}, \text{ls}_{b,3}, \text{tdec}_b\}_{b \in \{0,1\}}$.

Using the equivocal hiding of TC , we obtain

$$\begin{aligned} & \Pr \left[\beta = \tilde{\beta} \mid \begin{array}{l} \beta \leftarrow \{0, 1\} \\ (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.}\hat{\text{P}}_1(1^\lambda, \beta) \\ \tilde{\beta} \leftarrow \mathcal{V}^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(1^\lambda, \text{wi}_1, z) \end{array} \right] \\ & \leq \Pr \left[\beta = \tilde{\beta} \mid \begin{array}{l} \beta \leftarrow \{0, 1\} \\ (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.}\hat{\text{P}}'_1(1^\lambda, \beta) \\ \tilde{\beta} \leftarrow \mathcal{V}^{\mathcal{O}'(\text{st}, \text{aux}, \cdot)}(1^\lambda, \text{wi}_1, z) \end{array} \right] + \text{negl}(\lambda), \end{aligned}$$

where the oracle $\mathcal{O}'$ is defined as follows.

- $\mathcal{O}'(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$: If $|x_b| = \lambda_b$ and $(x_b, w_b) \in \mathbf{R}_\mathbf{L}$ for every $b \in \{0, 1\}$, return $\text{WIPOK.}\hat{\text{P}}'_2(\text{st}, \text{aux}, x_0, x_1, w_0, w_1, \text{wi}_2)$. Otherwise, return $\perp$.

Also, using the hiding of Com , we obtain

$$\Pr \left[\beta = \tilde{\beta} \mid \begin{array}{l} \beta \leftarrow \{0, 1\} \\ (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK.}\hat{\text{P}}'_1(1^\lambda, \beta) \\ \tilde{\beta} \leftarrow \mathcal{V}^{\mathcal{O}'(\text{st}, \text{aux}, \cdot)}(1^\lambda, \text{wi}_1, z) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Combining the above, we obtain the following as desired.

$$\Pr \left[\beta = \tilde{\beta} \mid \begin{array}{l} \beta \leftarrow \{0, 1\} \\ (\text{wi}_1, \text{st}, \text{aux}) \leftarrow \text{WIPOK}.\hat{\text{P}}_1(1^\lambda, \beta) \\ \tilde{\beta} \leftarrow \mathcal{V}^{\mathcal{O}(\text{st}, \text{aux}, \cdot)}(1^\lambda, \text{wi}_1, z) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

B Full Description of Hybrid Simulator-extractor $\mathcal{SE}'$

Hybrid simulator-extractor $\mathcal{SE}'$.

Simulation part. On input $(1^\lambda, z)$, the simulator-extractor $\mathcal{SE}'$ internally invokes $\mathcal{A}(1^\lambda, z)$ and emulates the left and right interactions for $\mathcal{A}$ in the following way.

In each right interaction, $\mathcal{SE}'$ interacts with $\mathcal{A}$ using the honest verifier algorithms. That is, $\mathcal{SE}'$ interacts with $\mathcal{A}$ in the i -th right interaction as follows.

Round i -1. Receiving $\widetilde{\text{tag}}_i$, compute $(\widetilde{\text{msg}}_{i,1}, \widetilde{\text{st}}_{V,i}) \leftarrow V_1(1^\lambda, \widetilde{\text{tag}}_i)$ and send $\widetilde{\text{msg}}_{i,1}$ to $\mathcal{A}$.

Round i -2. Receiving $\widetilde{\text{msg}}_{i,2}$, compute $\widetilde{\text{msg}}_{i,3} \leftarrow V_2(\widetilde{\text{st}}_{V,i}, \widetilde{\text{msg}}_{i,2})$ and send $\widetilde{\text{msg}}_{i,3}$ to $\mathcal{A}$.

In the left interaction, $\mathcal{SE}'$ interacts with $\mathcal{A}$ as follows.

Round 2. Receiving $(\text{tag}, \text{msg}_1)$, compute $(\text{msg}_2, \text{st}_P) \leftarrow P_1(1^\lambda, \text{tag}, \text{msg}_1)$ and send msg_2 to $\mathcal{A}$.

Round 4. Receiving (x, msg_3) , do the following.

4:1 Like the honest prover, check whether msg_3 is accepting, i.e., whether $\text{WIPOK}.\text{V}_D(x_{V,0}, x_{V,1}, \tau_{\text{wi}}) \stackrel{?}{=} 1$ for the WIPOK statements $x_{V,0}, x_{V,1}$ and transcript τ_{wi} defined based on $(\text{msg}_1, \text{msg}_2, \text{msg}_3)$. If the check fails, abort the left interaction.

4:2 If the check passes, do the following.

First left rewinding. Rewind $\mathcal{A}$ to the point just before sending msg_2 to $\mathcal{A}$ in Round 2. Then, sample a new Round-2 message msg'_2 using $P_1(1^\lambda, \text{tag}, \text{msg}_1)$ with fresh randomness, send msg'_2 to $\mathcal{A}$, and continue interacting with $\mathcal{A}$ until obtaining a new Round-3 message msg'_3 . (During the interaction with $\mathcal{A}$, the right interactions are simulated as before using fresh randomness.) Let $(x'_{V,0}, x'_{V,1})$ and τ'_{wi} denote the WIPOK statements and transcript defined based on $(\text{msg}_1, \text{msg}'_2, \text{msg}'_3)$.

Second left rewinding (same as above). Rewind $\mathcal{A}$ to the point just before sending msg_2 to $\mathcal{A}$ in Round 2. Then, sample a new Round-2 message msg''_2 using $P_1(1^\lambda, \text{tag}, \text{msg}_1)$ with fresh randomness, send msg''_2 to $\mathcal{A}$, and continue interacting with $\mathcal{A}$ until obtaining a new Round-3 message msg''_3 . (During the interaction with $\mathcal{A}$, the right interactions are simulated as before using fresh randomness.) Let $(x''_{V,0}, x''_{V,1})$ and τ''_{wi} denote the WIPOK statements and transcript defined based on $(\text{msg}_1, \text{msg}''_2, \text{msg}''_3)$.

If either msg'_3 or msg''_3 is not accepting, output fail and abort. Otherwise, restore the state of $\mathcal{A}$ to the one that $\mathcal{A}$ had before the above rewinding.

4:3 Compute $w_V := \text{WIPOK}.\text{Ext}((x'_{V,0}, x'_{V,1}), \tau'_{\text{wi}}, (x''_{V,0}, x''_{V,1}), \tau''_{\text{wi}})$. If $\exists \beta \in \{0, 1\}$ s.t. $(x'_{V,\beta}, w_V) \in \mathbf{R}_{\text{L}_V}$, parse w_V as (i-dec, o-dec) and let $w_{\text{td}} := (\beta, \text{i-dec})$. If there is no such β , output fail and abort.

4:4 Compute $\text{swi}_4 \leftarrow \text{SWIAOK}.\text{P}_2(\text{st}_{\text{swi}}, x_P, w_{\text{td}}, \text{swi}_3)$ by using w_{td} along with the SWIAOK prover state st_{swi} , statement x_P , and challenge swi_3 that are defined based on $(x, \text{msg}_1, \text{msg}_2, \text{msg}_3)$. Then, send $\text{msg}_4 := \text{swi}_4$ to $\mathcal{A}$.

Let *sim-view* be the joint view of $\mathcal{A}$ and the honest verifiers emulated in the above. We call the execution of $\mathcal{A}$ recorded in *sim-view* the *main thread*, and the executions of $\mathcal{A}$ generated during the left rewinding the *left look-ahead threads*. We call w_{td} (the witness used for SWIAOK in Step 4:4) the *trapdoor witness*. If the messages in the left and the i^* -th right interaction are not scheduled in Schedule 1 in the main thread, output fail and abort.

Extraction part. After completing the simulation part, $\mathcal{SE}'$ defines candidate witnesses $\{\tilde{w}_i\}_{i \in [m(\lambda)]}$ in the following way. For every $i \in [m(\lambda)] \setminus \{i^*\}$, $\mathcal{SE}'$ lets $\tilde{w}_i := \perp$. Also, $\mathcal{SE}'$ lets $\tilde{w}_{i^*} := \perp$ if the i^* -th right interaction is rejecting or uses the same tag as the left interaction. Otherwise, $\mathcal{SE}'$ defines $\tilde{w}_{i^*}$ as follows. Let $\tilde{x}_{i^*}$ and $\widetilde{\text{msg}}_{i^*,4}$ denote the statement and message sent by $\mathcal{A}$ in Round i^* -4.

$i^*:1$ Do the following.

Right rewinding. Rewind $\mathcal{A}$ to the point just before sending $\widetilde{\text{msg}}_{i^*,3}$ to $\mathcal{A}$ in Round i^* -3. Then, define a new Round-3 message $\widetilde{\text{msg}}'_{i^*,3}$ as follows: parse $\widetilde{\text{msg}}_{i^*,3}$ as $\widetilde{\text{msg}}_{i^*,3} = (\{\widetilde{i\text{-nm}}_{b,3}, \widetilde{o\text{-nm}}_{b,3}\}_{b \in \{0,1\}}, \widetilde{w}_3, \widetilde{sw}_3)$; then, sample a new challenge message $\widetilde{sw}'_3 \leftarrow \text{SWIAOK.V}_2(1^{\lambda_P})$ for SWIAOK and let $\widetilde{\text{msg}}'_{i^*,3} := (\{\widetilde{i\text{-nm}}_{b,3}, \widetilde{o\text{-nm}}_{b,3}\}_{b \in \{0,1\}}, \widetilde{w}_3, \widetilde{sw}'_3)$. Then, send $\widetilde{\text{msg}}'_{i^*,3}$ to $\mathcal{A}$ and continue interacting with $\mathcal{A}$ until obtaining a new Round 4 message $\widetilde{\text{msg}}'_{i^*,4}$ along with a new statement $\tilde{x}'_{i^*}$ in Round i^* -4. (During the interaction with $\mathcal{A}$, the left and other right interactions are simulated as before with the following exception. If EVENT_0 or EVENT_1 occurred in the main thread, then when Round 4 of the left interaction is simulated, Steps 4:2 and 4:3 are skipped, and Step 4:4 is executed by reusing the witness w_{td} that was used in the main thread.)

Let us call the interactions with $\mathcal{A}$ during the right rewinding the *right look-ahead thread*. If either of the following holds, output fail and abort.

- In the main thread, EVENT_0 or EVENT_1 occurs, and in the right look-ahead thread, either the new Round 4 message $\widetilde{\text{msg}}'_{i^*,4}$ is not accepting or w_{td} is not a valid witness for SWIAOK.
- In the main thread, EVENT_2 occurs, and in the right look-ahead thread, EVENT_2 does not occur.

$i^*:2$ Compute $\tilde{w}_{P,i} := \text{SWIAOK.Ext}(\tilde{x}_{P,i}, \tilde{\tau}_{\text{swi},i}, \tilde{x}'_{P,i}, \tilde{\tau}'_{\text{swi},i})$ for (i) the SWIAOK statement $\tilde{x}_{P,i}$ and transcript $\tilde{\tau}_{\text{swi},i}$ defined based on $(\tilde{x}_{i^*}, \widetilde{\text{msg}}_{i^*,1}, \widetilde{\text{msg}}_{i^*,2}, \widetilde{\text{msg}}_{i^*,3}, \widetilde{\text{msg}}_{i^*,4})$ and (ii) the SWIAOK statement $\tilde{x}'_{P,i}$ and transcript $\tilde{\tau}'_{\text{swi},i}$ defined based on $(\tilde{x}'_{i^*}, \widetilde{\text{msg}}_{i^*,1}, \widetilde{\text{msg}}_{i^*,2}, \widetilde{\text{msg}}'_{i^*,3}, \widetilde{\text{msg}}'_{i^*,4})$. If $(\tilde{x}_{P,i}, \tilde{w}_{P,i}) \in \mathbf{R}_{LP}$, let $\tilde{w}_{i^*} := \tilde{w}_{P,i}$. Otherwise, let $\tilde{w}_{i^*} := \perp_{\text{fail}}$.

Final output. After completing the extraction part, $\mathcal{SE}'$ outputs $(\text{sim-view}, \{\tilde{w}_i\}_{i \in [m(\lambda)]})$.

C Proof of Claim 4

Proof. First, we analyze the probability that $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_1$. By definition, $\text{BAD}_{i^*,1}$ occurs only when the i^* -th right interaction is scheduled in Schedule 1 in the main thread. Thus, aborting in the other cases does not decrease the probability that $\text{BAD}_{i^*,1}$ occurs. Therefore, $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_1$ with probability at least $\varepsilon_1 := 1/q(\lambda)$.

Next, we analyze the probability that $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_2$. Once the simulation part is completed, the steps defining $\tilde{w}_{i^*}$ are independent of the steps defining $\tilde{w}_i$ for any $i \neq i^*$. Thus, skipping the steps defining $\tilde{w}_i$ for every $i \neq i^*$ does not decrease the probability that $\text{BAD}_{i^*,1}$ occurs. Therefore, $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_2$ with probability at least $\varepsilon_2 := \varepsilon_1$.

Next, we analyze the probability that $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_3$. Since the expected number of the right look-ahead threads is 1, Markov's inequality implies that the number of the right look-ahead threads exceeds $2/\varepsilon_2$ with probability at most $\varepsilon_2/2$. Then, since each right look-ahead thread is generated identically, it follows that even when only a single right look-ahead thread is generated, $\text{BAD}_{i^*,1}$ occurs with probability at least $(\varepsilon_2 - \varepsilon_2/2) \cdot (2/\varepsilon_2)^{-1} = (\varepsilon_2/2)^2$. Therefore, $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_3$ with probability at least $\varepsilon_3 := (\varepsilon_2/2)^2$.

Next, we analyze the probability that $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_4$. Clearly, $\mathcal{SE}_4$ is identical to $\mathcal{SE}_3$ when neither EVENT_0 nor EVENT_1 occurs in the main thread. We therefore focus on the case that either EVENT_0 or EVENT_1 occurs in the main thread, and we show that when $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_3$, it also occurs in $\mathcal{SE}_4$. Let $\{\text{i-nm}_{b,1}\}_{b \in \{0,1\}}$ denote the inner NMC messages that are given in Round 1 of the left interaction, and let w_{td} and w'_{td} denote the witnesses that are used in Round 4 of the left interaction in the main and right look-ahead threads in $\mathcal{SE}_3$. Suppose $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_3$. By the definition of $\text{BAD}_{i^*,1}$, there exists $\beta \in \{0,1\}$ such that EVENT_β occurs both in the main and right look-ahead threads. Thus, w_{td} and w'_{td} can be parsed as $w_{\text{td}} = (\beta, \text{i-dec})$ and $w'_{\text{td}} = (\beta, \text{i-dec}')$ such that $\text{NMC.R}_{\text{D,weak}}(\text{i-nm}_{\beta,1}, \text{i-dec}) = 1$ and $\text{NMC.R}_{\text{D,weak}}(\text{i-nm}_{\beta,1}, \text{i-dec}') = 1$. Therefore, the unique decommitment candidate property of NMC implies $\text{i-dec} = \text{i-dec}'$, and thus, $w_{\text{td}} = w'_{\text{td}}$. Hence, using w'_{td} instead of w_{td} does not decrease the probability that $\text{BAD}_{i^*,1}$ occurs. Therefore, $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_4$ with probability at least $\varepsilon_4 := \varepsilon_3$.

Finally, we analyze the probability that $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_5$. Since the expected number of the left look-ahead threads is 2, Markov's inequality implies that the number of the left look-ahead threads exceeds $2 \cdot 2/\varepsilon_4$ with probability at most $\varepsilon_4/2$. Then, since each left look-ahead thread is generated identically, it follows that even when only two left look-ahead threads are generated, $\text{BAD}_{i^*,1}$ occurs with probability at least $(\varepsilon_4 - \varepsilon_4/2) \cdot (4/\varepsilon_4)^{-2} = 2 \cdot (\varepsilon_4/4)^3$. Therefore, $\text{BAD}_{i^*,1}$ occurs in $\mathcal{SE}_5$ with probability at least $\varepsilon_5 := 2 \cdot (\varepsilon_4/4)^3$. $\square$